

# Mergeable Replicated Data Types in Sal: Metatheory, the Embedded-Chain Family, and Sequential Specifications

Sal / FP Launchpad (KC + Claude)

July 17, 2026

one document consolidating three earlier notes (of July 7, July 14, and July 16–17, 2026)

## Abstract

Mergeable replicated data types (MRDTs) resolve concurrent divergence by a *three-way* merge `mergel( $l, a, b$ )`: the two divergent states  $a, b$  are reconciled relative to their lowest common ancestor  $l$  in a git-like version DAG. This document is the consolidated account of the Sal verification effort. Part I develops the mechanized metatheory: what an MRDT is and how to describe one; why the published soundness induction fails (a fully LCA-legal defeater) and why no lattice-style contract can replace it; the corrected architecture of canonical states and a ternary Join Lemma; the eight verification conditions that make it true, with their adequacy theorem and the discharged production catalogue; the conditioned generalization for state-dependent data types; and the factored discharge route (a generic honest-reachability metatheorem consuming per-configuration join lemmas) that the newest instances use, with the mergeable queue as its flagship. Part II presents the embedded-chain RGA, the repository’s canonical sequence datatype: a tombstone-free replicated list whose sort keys are immutable birth chains carried as entropy-coded coordinates, with the encoding contract and entropy law, kernel-checked RA-linearizability parametric in the code, the compaction theorem identifying its reads with the published tombstoned RGA’s, and the sided generalization that makes two-sidedness a parameter and hosts the Fugue/FugueMax arc. Part III records the sequential-specification campaign: for every production RDT in Sal, a kernel-checked theorem that the fold of any sequentially honest single-replica history equals the output of an independent naive sequential program, together with three machine-checked refutations giving a two-axis separation between do-faithfulness and merge-faithfulness. Everything stated as a theorem is mechanized in Lean 4 with zero `sorry`s in the cited chains; the negative results are machine-checked countermodels.

**How this document is organized.** The three parts are followed by a single consolidated open-questions section and a single mechanization map. Part I is the metatheory and can be read alone; Part II depends on Part I’s signature and its factored discharge route (§13); Part III depends on both, since its refutations calibrate what Part I’s convergence theorems do and do not certify. The document consolidates three earlier notes (the metatheory note of July 7, the embedded-chain design note of July 14, and the sequential-specification campaign note of July 16–17); reused material keeps its original wording, duplicated material appears once, and the previously separate open-question lists and mechanization maps are merged. One editorial rule governs the whole document: the *rehoming* tombstone-free RGA, an earlier design demoted after its sequential refutation, receives no narrative treatment here; it appears only as the campaign’s named countermodel (Part III) and in one-line contrasts where the separation needs the losing side.

# Contents

|           |                                                                               |           |
|-----------|-------------------------------------------------------------------------------|-----------|
| <b>I</b>  | <b>The metatheory</b>                                                         | <b>3</b>  |
| 1         | Introduction                                                                  | 3         |
| 2         | Mergeable replicated data types, by definition and example                    | 5         |
| 3         | The ternary setting                                                           | 7         |
| 4         | Three facts the metatheory must respect                                       | 8         |
| 4.1       | The LCA lemma is a reachability invariant — with a gap in the published proof | 8         |
| 4.2       | A fully LCA-legal execution defeats the soundness proof . . . . .             | 9         |
| 5         | Canonical states and the ternary Join Lemma                                   | 9         |
| 6         | Why the contract is a delta contract                                          | 11        |
| 7         | The eight verification conditions                                             | 12        |
| 8         | Adequacy                                                                      | 14        |
| 9         | The catalogue, discharged                                                     | 15        |
| 10        | The conditioned metatheorem                                                   | 16        |
| 11        | The conditioned verification conditions                                       | 16        |
| 12        | The metatheorem                                                               | 17        |
| 13        | The factored discharge route: honest reachability and per-configuration joins | 18        |
| <b>II</b> | <b>The embedded-chain family</b>                                              | <b>21</b> |
| 14        | The datatype: sort keys are birth chains                                      | 22        |
| 15        | The representation: chains as entropy-coded coordinates                       | 24        |
| 16        | The insert prefix: for the proof, and the proof alone                         | 33        |
| 17        | Validation                                                                    | 34        |
| 18        | The sided generalization: two-sidedness as a parameter                        | 35        |
| 19        | Comparison with Eg-walker                                                     | 38        |
| 20        | Related work                                                                  | 40        |
| 21        | Mechanization: the conditioned discharge and the compaction theorem           | 41        |

|                                                                    |    |
|--------------------------------------------------------------------|----|
| III Sequential specifications: convergence is not correctness      | 43 |
| 22 The specification gap                                           | 44 |
| 23 The obligation, and the rules of engagement                     | 45 |
| 24 The observation gap: positional interfaces, witnessed ops       | 45 |
| 25 The specifications, datatype by datatype                        | 46 |
| 25.1 No gap: the alphabets already coincide . . . . .              | 46 |
| 25.2 Gap: arbitration by stamp instead of program order . . . . .  | 46 |
| 25.3 Gap: overwrite by snapshot instead of “everything” . . . . .  | 47 |
| 25.4 Gap: dequeue() becomes delete-what-I-saw . . . . .            | 47 |
| 25.5 Gap: insert-at-index becomes insert-after-who-I-saw . . . . . | 47 |
| 25.6 Gap: one mark becomes a boundary pair . . . . .               | 48 |
| 26 The proofs: four tiers, three kinds of invariant                | 48 |
| 27 The refutations                                                 | 49 |
| 28 The two-axis separation                                         | 50 |
| 29 What composition does not give                                  | 50 |
| 30 The verified matrix                                             | 52 |
| IV Open questions and the mechanization map                        | 52 |
| 31 Open questions                                                  | 52 |
| 32 Mechanization map                                               | 56 |

## Part I

# The metatheory

## 1 Introduction

An MRDT execution looks like a git history: replicas fork, apply operations locally, and merge. Unlike a state-based CRDT, whose binary merge must reconcile two states with no memory of their common past, an MRDT’s merge receives a third argument — the state at the *lowest common ancestor* (LCA) of the two versions being merged — and may use it to compute *deltas*: what each side did since the fork. The paradigmatic example is the counter,

$$\text{mergeL}(l, a, b) = a + b - l,$$

which adds the two sides’ increments-since- $l$  instead of double-counting their common past (Figure 1).

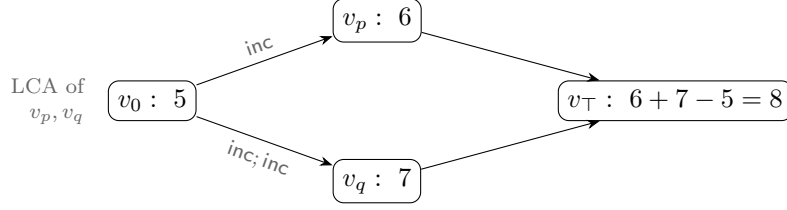


Figure 1: The counter MRDT. Both branches inherit the 5 from the fork point; the LCA argument lets the merge count each branch’s *delta* exactly once. No binary merge of 6 and 7 alone can recover 8: the information “how much is shared” lives in the DAG, and the LCA argument is how the data type reads it.

The Neem methodology (OOPSLA 2025) promises a clean division of labour: the data-type designer discharges a fixed catalogue of equations over `do`, `mergeL` and the conflict-resolution relation `rc` — mostly by SMT — and a soundness meta-theorem, proved once, converts those VCs into *RA-linearizability* of every reachable configuration. This part documents what a Lean 4 mechanization of that meta-theorem, in the full **ternary**, **version-DAG** setting, found: the meta-theorem’s central induction is unsound as written, and for LCA-sensitive data types several of the catalogue’s merge VCs are false in principle when quantified over all states. It also develops the corrected metatheory: a set-relative linearization order, canonical states  $\sigma(E)$  (each event set determines a unique state), and a ternary *Join Lemma* whose induction consumes a small, contextual VC surface. Its contributions:

1. **The paper’s LCA lemma is true but its proof has a gap** (§4.1):  $L(v_\ell) = L(v_1) \cap L(v_2)$  holds, but only as a *reachability invariant* of the store, whose proof needs a generator-uniqueness induction the paper silently assumes. We mechanize the transition system and the invariant.
2. **The soundness proof has a fully LCA-legal defeater** (§4.2): a reachable configuration of a legal add-wins MRDT — its final merge sitting at an honest LCA — on which every bottom-up peel of the paper’s derivation rules is stuck. The meta-theorem must be rebuilt, not patched.
3. **The lattice contract does not transfer; a delta contract does** (§6): the lattice laws are equations over *free* tuples — the wrong instances for an LCA-aware merge. Side-idempotence  $\text{mergeL}(l, a, a) = a$  with  $l$  free fails for the counter  $(2a - l)$ , but its only DAG-honest instance has  $l = a$  (the LCA of a version with itself *is* that version), where it holds trivially; and when two *distinct* histories reach equal side states,  $2a - l$  is the *correct* merge — both branches’ increments count — not an idempotence failure. The induced orders degenerate, and the honest associativity law has *four distinct LCAs* and holds only on feasible tuples. The moral: the algebra of an LCA-aware merge is an algebra of *causal histories*, not of concrete states. What replaces the lattice laws is a two-law *delta contract* — redistribution identities in which the LCA slot does, by arithmetic, the job idempotence does order-theoretically for CRDTs.
4. **Eight VCs suffice** (§7–§8): three relational VCs on `rc` (one of them carrying a different-replica guard — same-replica pairs are ordered by program order, and demanding `rc` cover them is unsatisfiable for real data types), merge symmetry, three feasibility-conditioned

delta laws, and one contextual *causal-delta* equation. The adequacy theorem is per *version*: every version in the store, LCAs included, linearizes.

5. **The VC set is validated on the production catalogue (§9):** the flat production MRDTs shipped in Sal — OR-set, an optimized OR-set, enable-wins flag, grow-only set and map, increment-only and PN counters, a tombstone RGA, Peritext, the multi-valued register and the add-wins priority queue — are discharged end-to-end, kernel-checked; the sequence data types with physical deletion go through the conditioned framework and its factored discharge route (§10–§13, Part II). The sweep produces an empirical *class map* of which data types need which strength of contract, with two surprises: tombstones are a metatheoretic trivializer, and commutativity class and delta class are independent axes.
6. **The metatheorem factors (§13):** conditioning splits as a generic honest-reachability metatheorem times a per-datatype join discharge (`HonestReach`, `JoinLemma3At`), with a uniform honesty shape (a client-checkable predicate holding of each event at the fold of its causal past). Three join species are observed in production (conditioned commutation, flat commutation, direct witness), and the mergeable queue, whose conflict graph admits no `rc` assignment at all, is the flagship of the direct-witness species: Peepul’s merge is its own linearization certificate. The same route carries Part II’s embedded-chain RGA.

Everything stated as a theorem below is mechanized with zero `sorry`s; §32 maps statements to Lean names and files (foundations under `Framework/`, metatheorems under `Metatheory/`, negative results under `Refutations/`, the per-RDT instances under `MRDT_Instances/`, the investigation record under `Development/`; each cited theorem kernel-checked).

## 2 Mergeable replicated data types, by definition and example

**Data type and implementation.** A replicated data type of *type*  $\tau = \langle O_\tau, Q_\tau, Val_\tau \rangle$  exposes a set of update operations  $O_\tau$ , a set of query operations  $Q_\tau$ , and a domain of query return values  $Val_\tau$ . An *MRDT implementation* of  $\tau$  is a tuple

$$\mathcal{D} = \langle \Sigma, \sigma_0, \text{do}, \text{mergeL}, \text{query}, \text{rc} \rangle,$$

where  $\Sigma$  is the space of replica states with initial state  $\sigma_0$ ;  $\text{do} : \Sigma \times \mathcal{E} \rightarrow \Sigma$  applies an *event* — an update-operation instance  $e = (t, r, o)$  carrying a globally unique timestamp  $t$ , its origin replica  $r$ , and the operation  $o \in O_\tau$ ;  $\text{mergeL} : \Sigma^3 \rightarrow \Sigma$  is the three-way merge, whose first argument is the state at the lowest common ancestor of the two versions being merged;  $\text{query} : \Sigma \times Q_\tau \rightarrow Val_\tau$  reads a state; and  $\text{rc} \subseteq O_\tau \times O_\tau$  is the *conflict-resolution* relation, consulted to order concurrent non-commuting operations (it is the data-type designer’s declaration of who wins a race). Queries never change state and play no role in the metatheory; the theory below reads the implementation as  $\langle \Sigma, \sigma_0, \text{do}, \text{mergeL}, \text{rc} \rangle$ .

**Describing an MRDT: the counter.** To describe an MRDT one gives the five components and, when operations can race, says who wins. The increment-only counter of Figure 1 is the smallest complete example:

$$\Sigma = \mathbb{Z}, \quad \sigma_0 = 0, \quad \text{do}(\sigma, (t, r, \text{inc})) = \sigma + 1, \quad \text{mergeL}(l, a, b) = a + b - l, \quad \text{rc} = \emptyset.$$

All operations commute, so  $\text{rc}$  is empty — yet the merge genuinely uses its LCA argument:  $a + b$  alone would double-count the history shared by the two branches, and the  $-l$  subtracts it exactly once. This “read the shared past off the LCA” pattern recurs throughout the catalogue.

**A data type with races: the OR-set.** The add-wins observed-remove set stores tagged elements: **add** inserts a tag unique to the adding event, **rem** deletes exactly the tags it has *seen*. Sequentially, a remove after an add deletes it; concurrently the designer declares the add the winner:  $\text{rem} \xrightarrow{\text{rc}} \text{add}$ . The merge keeps a tag iff both sides have it (nobody deleted it) or some side added it since the fork:

$$\text{mergeL}(l, a, b) = (a \cap b) \cup (a \setminus l) \cup (b \setminus l).$$

The LCA here plays the role a tombstone set plays for the CRDT OR-set: deletion is absence *relative to l*, and the state needs no graveyard.

**The generalized signature.** Both examples’ operations make sense on *every* state: any state can be incremented, any tag can be added. Such data types are *flat*, and for them the metatheory of §3–§9 quantifies its VCs over all states. But a data type whose operations carry *preconditions* — insert under a node that must exist, delete a node that must be live — breaks that quantification: its commutation equations are simply false at nonsense states. The full signature therefore extends the tuple with three declarations (**ConditionedMRDTSig**; formalized in §11):

$$\mathcal{D} = \langle \Sigma, \sigma_0, \text{do}, \text{mergeL}, \text{rc}, \text{Inv}, \text{app} \rangle \text{ together with } \approx \subseteq \Sigma \times \Sigma,$$

where **Inv** is a state invariant (a shape over-approximation of reachability), **app** an applicability predicate (when an event is sensible at a state), and  $\approx$  an observational equivalence (what queries can distinguish; representation residue is quotiented away). Commutation is then required only at **Inv**-states where both events are applicable. Flat data types take  $\text{Inv} = \text{app} = \top$  and  $\approx = =$ , collapsing the general signature to the plain one — one framework, of which §3–§9 is the flat instance and §10 onwards the general case.

**The running hard example: sequences with physical deletion.** The datatype class that forces the general signature is the replicated list underlying collaborative text editing. Characters are nodes identified by timestamps, each inserted *after* an anchor node; production RGAs keep deleted nodes as *tombstones* because later concurrent inserts may still anchor on them, and a *tombstone-free* design deletes physically. Its operations are precondition-laden: an insert names an anchor that must exist, a delete names a live node, and the flat update layer quantifies its commutation VCs over every state, including nonsense states no execution produces, where those equations are false. Worse, what makes reachable-state commutation true is a property of the *execution that generated the operation* (the issuer saw the anchor live), not of the state the operation later meets, so it is not expressible as a  $\forall \sigma$  equation over the signature alone. This is the reason the general signature exists: **Inv** and **app** name the sensible states and operations, an honesty discipline makes generation-time facts available to the proofs, and  $\approx$  absorbs representation residue. Part II presents the embedded-chain RGA, the sequence datatype that holds the repository’s canonical seat, together with its end-to-end discharge through the factored route of §13; a predecessor design that carried recorded ancestor paths and rehomed orphans on delete was proved RA-linearizable in the same framework and was later demoted by its sequential refutation (Part III).

### 3 The ternary setting

This section fixes the model: the data-type signature, the replicated store and its transitions, and the specification the metatheorem must deliver. The definitions are the paper’s, with one correction flagged where it occurs.

**Signature.** An MRDT is  $\langle \Sigma, \sigma_0, \text{do}, \text{mergeL}, \text{rc} \rangle$  as in §2, read flat for now ( $\text{Inv} = \text{app} = \top$ ,  $\approx = =$ ; the general case resumes in §10). We write  $e(\sigma)$  for  $\text{do}(\sigma, e)$ , and  $e_1 \rightleftharpoons e_2$  when  $e_1(e_2(\sigma)) = e_2(e_1(\sigma))$  for every  $\sigma$ .

**Executions.** A configuration carries a *store*: a DAG of versions, each holding a state and the set of events that produced it, with per-replica heads. Transitions fork a replica, apply a fresh event at a head, query, or **merge**: pick heads  $v_1, v_2$ , find their LCA  $v_\ell$  in the DAG, and install a new version with state  $\text{mergeL}(\sigma_{v_\ell}, \sigma_{v_1}, \sigma_{v_2})$  and event set  $L(v_1) \cup L(v_2)$ . Visibility  $\xrightarrow{\text{vis}}$  records what each event’s issuing replica had seen.

**RA-linearizability, per version.** Write  $e_1 \parallel e_2$  when neither  $e_1 \xrightarrow{\text{vis}} e_2$  nor  $e_2 \xrightarrow{\text{vis}} e_1$  (concurrency).

**Definition 3.1** (linearization order, set-relative). *For an event set  $E$ , the order  $\text{lo}^E$  puts  $e_1$  before  $e_2$  iff*

$$(e_1 \xrightarrow{\text{vis}} e_2 \wedge e_1 \not\rightleftharpoons e_2) \vee (e_1 \parallel e_2 \wedge e_1 \xrightarrow{\text{rc}} e_2 \wedge \neg \exists e_3 \in E. e_2 \xrightarrow{\text{vis}} e_3 \wedge e_2 \not\rightleftharpoons e_3) :$$

*a visible non-commuting pair is ordered by visibility; a concurrent pair whose conflict  $\text{rc}$  resolves is ordered by  $\text{rc}$  — unless  $e_2$  is already absorbed (overwritten) by a later non-commuting event of  $E$ . The paper’s order  $\text{lo}$  is the variant whose absorber clause ranges over the whole configuration; since a  $\text{lo}$ -edge asserts no absorber anywhere,  $\text{lo} \subseteq \text{lo}^E$  pointwise, for every  $E$ .*

**Definition 3.2** (RA-linearizability). *A list  $\pi = e^1 e^2 \dots e^n$  witnesses a version holding state  $s$  and event set  $E$  when (i)  $\pi$  enumerates  $E$  without repetition; (ii)  $\pi$  extends  $\text{lo}$ :  $i < j$  implies  $\neg(e^j \text{ lo } e^i)$ ; and (iii)  $\pi$  folds to the state:  $e^n(\dots e^2(e^1(\sigma_0))\dots) = s$ . A configuration is RA-linearizable when every version in the store — not only the replica heads; LCAs are historical versions and the merge reads them — admits a witness. In the general signature the fold clause (iii) is read up to the data type’s observational equivalence,  $e^n(\dots e^1(\sigma_0)\dots) \approx s$  — the same definition with its equality parameter generalized; flat data types instantiate  $\approx = =$  and this section’s form is exactly that instance.*

*IsRALinearizable3, IsRALinearizable3Eq*

Why a set-relative order? One definitional correction is forced at the outset: for every internal use, the absorber clause must range over the event set of *the version being linearized*, not over the whole configuration — an event can gain absorbers from another branch, silently cancelling an order edge inside a version whose own state depends on it (the defeater of §4.2 realizes exactly this). We therefore construct witnesses respecting  $\text{lo}^E$  throughout; since  $\text{lo} \subseteq \text{lo}^E$ , every such witness also extends the paper’s order, and Definition 3.2 is delivered exactly as the paper states it.



**Running examples.** Three data types exercise every corner of the theory:

- the **counter**:  $\Sigma = \mathbb{Z}$ ,  $\text{inc}(\sigma) = \sigma + 1$ ,  $\text{mergeL}(l, a, b) = a + b - l$ ; all operations commute, but the merge genuinely uses its LCA;
- the **OR-set** (tagged add-wins set): elements are tagged by the adding event’s timestamp;  $\text{rem}$  deletes the tags it has seen; concurrently,  $\text{rem} \xrightarrow{\text{rc}} \text{add}$  resolves in favour of the add. The merge keeps an element iff *both* sides have it or *some* side added it since the fork:

$$\text{mergeL}(l, a, b) = (a \cap b) \cup (a \setminus l) \cup (b \setminus l)$$

— pointwise, “if the tag is in  $l$ , require it in both branches (nobody deleted it); otherwise one branch suffices (somebody added it).” Here the LCA acts as a *tombstone set*: deletion is represented by *absence relative to  $l$* , so the state itself needs no graveyard;

- the **enable-wins flag**: per-replica pairs (counter, flag); enabling bumps your own counter and sets your flag; disabling clears all flags; the merge compares each replica’s counter against the LCA’s to decide whether an enable “happened since the fork” on that replica. Its merge *compares* counters rather than accumulating sets — and that difference will be visible in the metatheory (§9).

## 4 Three facts the metatheory must respect

Before building anything, we record two boundary facts about the paper’s development: its store-level LCA lemma is true but under-proved, and its soundness induction is refuted by a fully legal execution. Together they fix the shape of the corrected metatheory — resting on a mechanized store invariant (§4.1) and built around the join rather than the sides (§4.2).

### 4.1 The LCA lemma is a reachability invariant — with a gap in the published proof

Everything above silently used:

**Lemma 4.1** (LCA lemma). *In every reachable configuration, if  $v_\ell$  is an LCA of versions  $v_1, v_2$  in the store’s DAG, then  $L(v_\ell) = L(v_1) \cap L(v_2)$ .*

This is what lets merge VCs be stated over *event sets*: the state the merge receives as  $l$  is the canonical state of exactly the intersection of the two sides’ histories. The lemma is true — but it is a *global induction over the reachable store*, not a local fact. Its mechanized proof threads a store invariant (all parents allocated; event sets monotone along edges; and an *origin* invariant — each event appears first at a unique generator version) through every transition; the published proof asserts the generator-uniqueness step without the induction that sustains it. An erratum-sized gap: the statement survives, the proof as written does not. (A related boundary fact: *criss-cross* merges can defeat the *existence* of an LCA — they gate merge-enabledness, not the lemma. Open Question 31.5 takes up the extension.)



## 4.2 A fully LCA-legal execution defeats the soundness proof

The generic half of the paper’s contract is a *bottom-up linearization* theorem: given linearization witnesses for the two sides of a merge, construct one for the merged version by repeatedly *peeling* a final event through `mergeL` using the catalogue’s interchange VCs. One might hope the version DAG’s discipline — every merge sitting at an honest LCA — keeps the peel supplied with peelable events. It does not: one staging replica suffices to realize the intersection of two histories as an honest version, and the merge induction is stuck.

**Example 4.2** (the defeater). *The data type is a one-key add-wins skeleton with tagged adds:  $\sigma = (A, D) \in 2^{\mathbb{T}} \times 2^{\mathbb{T}}$  with live set  $A \setminus D$ ; `addt` inserts its tag into  $A$ ; `rem` kills everything it currently sees ( $D := A \cup D$  — the state-dependence is what makes `add`  $\not\equiv$  `rem`); concurrently `rem`  $\xrightarrow{rc}$  `add` (add wins); the merge is componentwise union, ignoring its LCA argument. A legal MRDT. Replica  $p$  adds ( $A_p$ ), replica  $q$  adds ( $A_q$ ). A staging replica  $s$  merges both one-add versions, creating  $v_s$  with  $L(v_s) = \{A_p, A_q\}$ . Now  $p$  removes ( $R_p$ , seeing only  $A_p$ ) and then merges with  $v_s$ ; symmetrically  $q$  removes ( $R_q$ ) and merges with  $v_s$ . Figure 2 shows the DAG. The final merge of the two heads finds  $LCA = v_s$ , and  $L(v_s) = \{A_p, A_q\} = L(head_p) \cap L(head_q)$  — the configuration is fully legal. Now count last events.  $p$ ’s head lives on exactly  $A_q$ ’s tag, so a state-correct witness must run  $R_p$  before  $A_q$ , and neither  $R_p$  nor  $A_p$  can come last: every linearization of  $p$ ’s head ends in  $A_q$ . Symmetrically, every linearization of  $q$ ’s head ends in  $A_p$ . Yet in the merged version every tag is dead, so its linearizations must end in one of the removes. The events the induction must peel are buried in the very sides that own them; no ordering of the paper’s bottom-up rules applies.*

The two removes commute (`rem`  $\not\equiv$  `add` but `rem`  $\rightleftharpoons$  `rem`), which invites a particular rescue attempt: the paper’s BOTTOMUP-2-OP rule (`inter_lca_2op`) admits the commuting case and could, one hopes, peel a remove last. It cannot, for a reason that is a one-line state fact. To peel  $o$  last from a side, that side’s state must be an  $o$ -image — of the form  $do(\sigma', o)$  with  $o$  outermost. But in the add-wins skeleton *every* `rem`-output has empty live set ( $rem(A, D) = (A, A \cup D)$ ), whereas both heads carry a live tag ( $head_p$  lives on  $A_q$ ,  $head_q$  on  $A_p$  — the opposite add, merged in *after* the local remove). So neither head is a `rem`-image, and no bottom-up rule can extract a remove from it. The tempting linearization  $[A_p, R_p, A_q, R_q]$  is a valid witness of the *merged* version, but its restriction to  $head_q$  folds to the all-dead state, not  $head_q$ ’s actual state (live  $A_p$ ): it is not assembled from a state-correct side witness, which is exactly what the induction requires. All three facts are mechanized (`InterLca2op_Defeater_Arbiter: awset_rem_output_empty, no_inter_lca_2op_rem_peel_of_defeater, crack1_witness`).

The consequence for the design: the metatheorem cannot be patched peel-by-peel — it must be rebuilt on canonical states and a Join Lemma, with merge-side VCs that are statements *about the join*, not about how either side’s state factors.

## 5 Canonical states and the ternary Join Lemma

We now rebuild — new proof architecture, and with it a *new set of verification conditions* to replace the paper’s catalogue. The lesson of the defeater is architectural: witnesses for a merged version cannot be manufactured out of witnesses for its sides, so the corrected metatheory must not traffic in witness lists at all. The rebuild has two layers. The *update layer* makes states, not sequences, the objects of the induction: it assigns to every event set  $E$  a single well-defined state, its *canonical state* (Definition 5.2 below). The *merge layer* is then one equation between

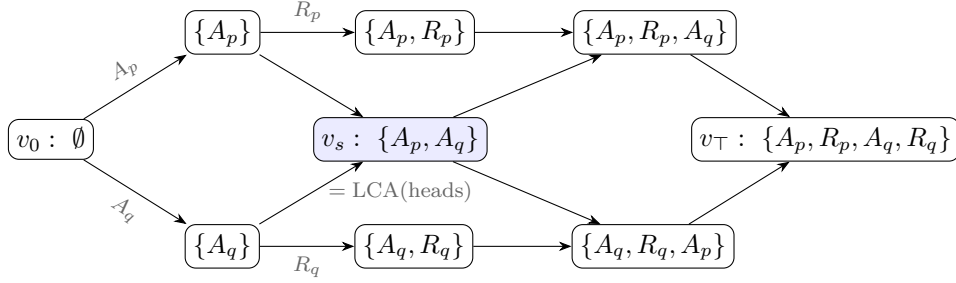


Figure 2: The defeater (Example 4.2). Nodes are versions labelled by their event sets; the shaded version  $v_s$ , created by a staging replica, realizes  $L(\text{head}_p) \cap L(\text{head}_q)$  as an honest store version, so the final merge is LCA-legal. The removes  $R_p, R_q$  — the only events a bottom-up derivation may peel last — are buried mid-history on their own sides.

canonical states — the ternary *Join Lemma* — and establishing it from per-data-type VCs becomes the entire burden of the metatheorem (§6–§7).

The update layer comes first, and a structural observation makes it cheap: this half of the corrected theory never mentions the merge. Its one theorem is stated over the update signature  $\langle \Sigma, \sigma_0, \text{do}, \text{rc} \rangle$  alone:

**Theorem 5.1** (convergence). *Assume the update-layer VCs (items 1–3 of §7). Then any two  $\text{lo}^E$ -respecting enumerations of a finite event set  $E$  fold from  $\sigma_0$  to the same state.*

This is where the set-relative reading of §3 earns its keep: with the paper’s configuration-wide order in place of  $\text{lo}^E$ , the theorem is *false* — the defeater’s heads are already counterexamples, since two order-respecting replays of the three events of  $\text{head}_p$  reach different states. Convergence licenses a definition:

**Definition 5.2** (canonical state). *For a finite event set  $E$ , the canonical state  $\sigma(E)$  is the state obtained by folding any  $\text{lo}^E$ -respecting enumeration of  $E$  from  $\sigma_0$ . At least one respecting enumeration exists because  $\text{lo}^E$  is acyclic (VC 2); the result is independent of the choice by Theorem 5.1; and  $\sigma(\emptyset) = \sigma_0$ . We write  $\sigma(E)$  for the canonical state of an event set, and  $\sigma_v$  for the state a version  $v$  happens to hold in the store — proving that the two coincide is precisely the metatheorem’s job.*

With canonical states in hand, witness lists disappear as promised: a version linearizes iff  $\sigma_v = \sigma(L(v))$  — the witness, when one is owed, is any respecting enumeration of  $L(v)$ . The induction can therefore maintain a state-level invariant (“every version holds  $\sigma$  of its events”) instead of constructing sequences. The one genuinely ternary statement is what the merge must produce:

**Definition 5.3** (ternary Join Lemma). *For backward-closed event sets  $E_1, E_2$  of a reachable configuration,*

$$\sigma(E_1 \cup E_2) = \text{mergeL}(\sigma(E_1 \cap E_2), \sigma(E_1), \sigma(E_2)).$$

By Lemma 4.1 the first argument is exactly the state the store hands the merge. The adequacy proof (§8) is then an induction over the transition system maintaining “every version holds the canonical state of its event set”; the merge case is the Join Lemma. What remains is the question this part exists to answer: which per-data-type VCs make the Join Lemma true?

## 6 Why the contract is a delta contract

Where should the laws that prove the Join Lemma come from? The obvious supplier is the CRDT tradition. For state-based CRDTs — binary merge, no LCA — merge-coherence is lattice-flavoured: associativity, idempotence, inflation of updates. None of the three transfers to the ternary merge: their free-tuple forms are the wrong equations for an LCA-aware merge, and their honest instances are too weak to power a lattice-style proof. This section works through why, because the shape of what fails to transfer reveals the contract that succeeds.

**Idempotence: the free-tuple form is the wrong equation.** The *diagonal* law  $\text{mergeL}(a, a, a) = a$  — the paper’s own MERGEIDEMPOTENCE — holds (for the counter:  $a + a - a = a$ ), but it is not the law a lattice-style proof consumes. The binary architecture uses idempotence to absorb a duplicated *side*,  $\text{merge}(a, a) = a$ , with nothing tying the two copies to a common past; the ternary analogue would be  $\text{mergeL}(l, a, a) = a$  with  $l$  *free*. But quantifying  $l$  freely asks the merge about tuples the version DAG never produces. Merging a version with *itself* has  $l = a$  — the LCA of  $a$  and  $a$  is  $a$  — where the law collapses to the diagonal and holds. And when two *distinct* histories happen to reach equal side states, feasibility puts  $l$  strictly below them and the counter’s  $\text{mergeL}(l, a, a) = 2a - l$  is the *correct* answer: both branches’ increments count. So there is no idempotence *failure* — there is no meaningful side-idempotence *law*: its honest instances are the trivial diagonal, and demanding it unconditionally would exclude precisely the LCA-sensitive data types the ternary theorem exists for.

The general principle, which the rest of this section instantiates: the algebraic properties of an LCA-aware merge are properties of the *causal history*, not of the concrete state. Two sides merge idempotently when their *histories* coincide — which is what forces  $l = a$  — not when their states happen to be equal; equal states with different histories rightly merge differently. The lattice laws go wrong exactly by quantifying over states while forgetting the histories that produced them, and every law that survives below is stated over history-indexed data: canonical states  $\sigma(E)$ , feasible tuples (the LCA slot carrying the intersection history), downset deltas.

**No usable order.** The lattice-style workhorse order  $x \sqsubseteq y := \text{merge}(x, y) = y$  has ternary candidates  $x \sqsubseteq_l y := \text{mergeL}(l, x, y) = y$ , but for the counter this relation degenerates to  $x = l$ ; no antisymmetry survives to power order-theoretic reasoning. The single contextual VC below is accordingly an *equation*, not an inequality.

**Associativity is the wrong target.** The natural guess —  $\text{mergeL}(l, \text{mergeL}(l, a, b), c) = \text{mergeL}(l, a, \text{mergeL}(l, b, c))$  — assumes one LCA for all three merges. In an execution, re-associating a three-way reconciliation involves *four* distinct LCAs:

$$\text{mergeL}(l_{(ab)c}, \text{mergeL}(l_{ab}, a, b), c) \stackrel{?}{=} \text{mergeL}(l_{a(bc)}, a, \text{mergeL}(l_{bc}, b, c)),$$

with  $l_{ab} = \sigma(E_a \cap E_b)$ ,  $l_{(ab)c} = \sigma((E_a \cup E_b) \cap E_c)$ , and so on. For the counter — whose canonical states are additive measures over event sets — the two sides come to  $a + b + c$  minus the two LCA measures, and they agree because

$$(E_a \cup E_b) \cap E_c = (E_a \cap E_c) \cup (E_b \cap E_c) \implies \text{both LCA-sums} = |E_{ab}| + |E_{ac}| + |E_{bc}| - |E_{abc}|$$

by inclusion–exclusion. The law holds, but only *through the set-algebra of intersections*: over four unconstrained states it is false. Honest associativity is inherently a feasible-tuple fact —

and, the mechanization’s pleasant surprise, **the corrected induction never needs it**. Every merge the induction forms already sits at its honest LCA; the only laws consumed are two *redistribution* identities that equate two honestly-formed merges:

**Definition 6.1** (the delta contract).

$$\begin{aligned} \text{LOCAL-REDISTRIBUTE: } \text{mergeL}(l, \text{mergeL}(m, x, c), y) &= \text{mergeL}(m, \text{mergeL}(l, x, y), c), \\ \text{REDISTRIBUTE: } \text{mergeL}(\text{mergeL}(m, x_0, c), \text{mergeL}(m, x_1, c), \text{mergeL}(m, x_2, c)) \\ &= \text{mergeL}(m, \text{mergeL}(x_0, x_1, x_2), c). \end{aligned}$$

Read  $c$  as a *delta relative to  $m$*  (in the induction,  $c$  will be “apply event  $e$  to its causal past”). LOCAL-REDISTRIBUTE says a delta application commutes past an enclosing merge through one branch slot. REDISTRIBUTE says a delta carried by *all three* components extracts *once* — the LCA slot cancels the duplicates by arithmetic. This is the delta contract’s quiet coup: cancelling the duplicated shared contribution is exactly the job idempotence performs in lattice-based convergence arguments, now done without any idempotence — which is why the counter sails through (REDISTRIBUTE for the counter is the identity  $\sum x_i + 3c - 3m - (x_0 + c - m) - \dots$  collapsing on both sides).

**Where the contract holds unconditionally — and where it cannot.** The two laws hold *on all states* exactly for the group-like class (the counter) and the lattice-like, LCA-blind class (grow-only structures). They are *unconditionally false* for the OR-set: take an element tag present in  $c$  and  $l$  but in neither branch — the two sides of LOCAL-REDISTRIBUTE then disagree about whether the LCA’s copy survives. But such a tuple can never arise: a tag in a canonical LCA state is in *both* branches unless removed, and a removal is itself an event of the branch. The failing tuples are *infeasible*, and this is a theorem-grade phenomenon, not an inconvenience:

**Finding.** *For LCA-sensitive MRDTs beyond the group  $\oplus$  lattice classes, the merge-coherence laws are irreducibly feasibility-conditioned: true on canonical tuples at honest LCAs, false in general. Quantifying merge VCs over all states — the published catalogue’s style — is not merely inconvenient for such data types; for several laws it is wrong in principle. (The enable-wins flag even falsifies the unit law  $\text{mergeL}(\sigma_0, \sigma_0, s) = s$  on an infeasible state with a set flag and a zero counter.)*

Accordingly the delta laws enter the final contract in feasibility-conditioned form: quantified over canonical states of backward-closed event sets, with every `mergeL` node at its honest LCA. The unconditional form remains available as a one-line *on-ramp* (it trivially implies the conditioned form) for the group and lattice classes.

## 7 The eight verification conditions

The update-side obligations of §5 and the merge-side laws of §6 now assemble into the full contract — a *new* VC set, replacing the paper’s catalogue of twenty-four. For an MRDT  $\langle \Sigma, \sigma_0, \text{do}, \text{mergeL}, \text{rc} \rangle$ , the contract is:

**Update layer (unconditional, SMT-shaped).**

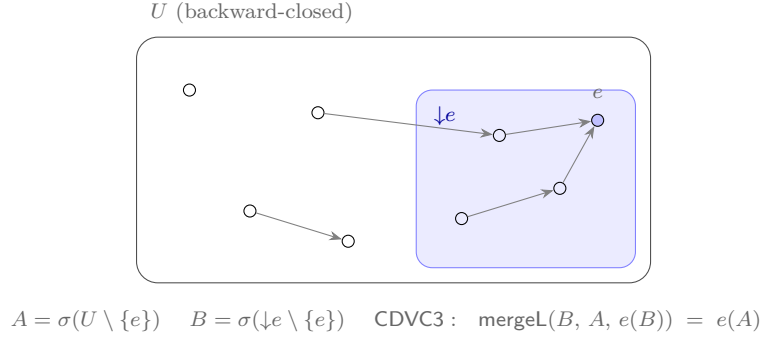


Figure 3: The causal-delta equation. The shaded region is  $e$ 's causal past  $\downarrow e$  (what  $e$  saw);  $e$  is  $\text{lo}^U$ -maximal.  $e$ 's effect on the whole world  $A$  equals: compute  $e$ 's effect on its own causal past  $B$ , then merge that delta into  $A$  over LCA  $B$ . The merge sits at its honest LCA:  $(U \setminus \{e\}) \cap \downarrow e = \downarrow e \setminus \{e\}$ . Concurrent context that  $e$  never observed cannot amplify what  $e$  does.

1. **rc-non-comm (guarded)** — for distinct events *from different replicas*: they fail to commute iff rc orders them in exactly one direction. *The conflict table is the commutativity table, with a direction.* The different-replica guard is a semantic necessity, not a convenience: same-replica events are totally ordered by program order, so they are never concurrent and there is no conflict for rc to resolve. An unguarded VC would instead demand that rc order *every* non-commuting pair — unsatisfiable for real data types: the *optimized OR-set*'s same-replica same-element adds do not commute (the newer add evicts the older tag), yet they can never race, and its rc rightly ignores them. (The paper's  $F^*$  artifact carries exactly this guard in its interface — `App_mrdt.fsti: requires distinct_ops o1 o2 /\ get_rid o1 <> get_rid o2.`)
2. **no-rc-chain** — conflict priorities do not chain: never  $o_1 \xrightarrow{\text{rc}} o_2$  and  $o_2 \xrightarrow{\text{rc}} o_3$ . This is what makes the linearization order acyclic, so maximal events exist — the entire peel-and-recurse architecture stands on it.
3. **cond-comm** — a resolved conflict is invisible once an absorber runs: swapping an rc-ordered concurrent pair deep in a fold changes nothing for any later non-commuting event, across arbitrary intervening events. This is the engine of convergence.

#### Merge layer.

4. **merge symmetry** —  $\text{mergeL}(l, a, b) = \text{mergeL}(l, b, a)$ .
5. **feasible unit** —  $\text{mergeL}(\sigma_0, \sigma_0, \sigma(E)) = \sigma(E)$ : merging over nothing is a no-op, on states that can arise.
6. **feasible local-redistribute** and
7. **feasible redistribute** — Definition 6.1, quantified over canonical tuples at honest LCAs.
8. **the causal-delta equation (CDVC3)** — for a  $\text{lo}^U$ -maximal event  $e$  of a backward-closed  $U$ , with  $A = \sigma(U \setminus \{e\})$  and  $B = \sigma(\downarrow e \setminus \{e\})$  the canonical state of  $e$ 's own causal past:

$$\text{mergeL}(B, A, e(B)) = e(A).$$

What an event does is determined by what it saw (Figure 3): applying  $e$  to the whole world equals computing  $e$ ’s delta on its causal past and merging it in over that past.

Items 1–4 are one-liners for every data type we have discharged; items 6–7 frequently hold unconditionally anyway (for the OR-set, REDISTRIBUTE is a pointwise Boolean tautology); essentially the entire per-data-type proof effort concentrates in item 8, whose discharge follows a uniform recipe: characterize  $\sigma(E)$  (a “sandwich” invariant along canonical enumerations), then a *trichotomy* placing every event the maximal  $e$  interacts with either inside  $\downarrow e$  or absorbed on a side that owns it.

Compare the published catalogue: 24 VCs per data type, plus five more the soundness proof uses silently. Seventeen of the 24 — the BOTTOMUP induction-scheme families — exist to drive the broken induction and are consumed nowhere in the corrected one; the paper’s MERGEIDEMPOTENCE is likewise never used. The seventeen were, however, doing honest work of a different kind: they are an *automation layer*, Skolemizing “holds on feasible states” into base-and-step equations an SMT solver can discharge. Rebuilding that layer, aimed at the corrected target (CDVC3 rather than BOTTOMUP), is in our view the most valuable open engineering problem this work leaves (§31).

## 8 Adequacy

The contract delivers. Everything a data type owes is the eight equations of §7; everything else is proved once, for all data types, as part of the execution model.

**Theorem 8.1** (soundness of the eight VCs). *If an MRDT satisfies VCs 1–8, then every configuration reachable from the initial configuration in the ternary transition system is RA-linearizable, per version.*

*ra\_linearizable\_of\_core\_feasible\_cd3*

The proof shape: the update layer powers the set-relative convergence theorem, hence canonical states; the transition induction maintains “every store version is canonical for its event set” (fork and query are trivial; apply is the extend lemma); the merge case reduces, via Lemma 4.1, to the ternary Join Lemma, which is proved from VCs 4–8 by strong induction on  $|E_1 \cup E_2|$ : select a  $\text{lo}^{E_1 \cup E_2}$ -maximal  $e$ , decompose each side containing  $e$  through  $\downarrow e$  (LOCAL-REDISTRIBUTE at the honest LCA  $\downarrow e \setminus \{e\}$ ), cancel the shared delta (REDISTRIBUTE), close the principal step with CDVC3, and recurse. The buried-event pathology of Example 4.2 dissolves structurally: no step ever asks how a *side*’s state factors — every factoring happens through the join.

Everything else one might fear owing is proved once, for all data types, as an execution-model invariant: visibility transitivity, per-version backward closure, store well-formedness, and Lemma 4.1 itself. The data type owes exactly the eight equations.

**A second route, and an honest asymmetry.** One discharged data type does not fit the pattern above. The enable-wins flag’s merge *compares counters* against the LCA, and for such merges the Join Lemma’s hypotheses as stated (backward closure under non-commuting-visibility) are provably too weak: there is a closure-legal tuple — a live LCA enable, a dead post-LCA enable inflating one side’s counter, the killing disable visible only to the other side — on which the production merge computes the wrong flag. Full *causal* closure (which the store invariant actually supplies) excludes it, and the flag is discharged through a full-closure variant of the join. The finding: **required closure strength is a function of the merge’s**

| MRDT                           | contract tier | end-to-end theorem             | $\alpha$ -characterization (one line)                                                                                                                                                     |
|--------------------------------|---------------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Grow-only set                  | unconditional | GDSet_ra_linearizable3_eq      | union of adds                                                                                                                                                                             |
| Grow-only map                  | unconditional | GDMap_ra_linearizable3_eq      | union of $(k, v)$ records                                                                                                                                                                 |
| Inc-only counter               | unconditional | IDC_ra_linearizable3_eq        | #events                                                                                                                                                                                   |
| PN-counter                     | unconditional | PN_ra_linearizable3_eq         | #inc - #dec                                                                                                                                                                               |
| RGA (tombstone)                | unconditional | RGM_ra_linearizable3_eq        | grow-only nodes + tombstones                                                                                                                                                              |
| Peritext                       | unconditional | Peritext_ra_linearizable3_eq   | grow-only mark records                                                                                                                                                                    |
| OR-set                         | feasible      | ORSet_ra_linearizable3_eq      | tag live iff added, no vis-later rem                                                                                                                                                      |
| OR-set (efficient)             | feasible      | ORSetE_ra_linearizable3_eq     | - or same-replica add later (eviction)                                                                                                                                                    |
| Enable-wins flag               | full-closure  | EWFlag_ra_linearizable3_eq     | per key: $ctr \neq \#enables$ ; flag iff unskilled enable                                                                                                                                 |
| Multi-valued register          | feasible      | MVR_ra_linearizable3_eq        | tagged writes $\times$ overwrite log; all-comm $\Rightarrow B \rightarrow a_0$                                                                                                            |
| Add-wins priority queue        | feasible      | AWPQ_ra_linearizable3_eq       | the OR-set pattern on A; grow-only I                                                                                                                                                      |
| Bounded counter (ecrow)        | all-comm      | BC_ra_linearizable3_eq         | convergence flat; the bound is a conditioned safety theorem (bc_version_inv)                                                                                                              |
| FWW reservation register       | all-comm      | FWW_ra_linearizable3_eq        | payload arbitration (min-ts semilattice); winner characterized at every version (fww_version_min)                                                                                         |
| LWW register                   | all-comm      | LWW_ra_linearizable3_eq        | payload arbitration (max-ts semilattice); the last-writer twin of FWW; winner characterized (lww_version_max); end-to-end mechanized, unlike the flat CRDT LWW (sorry-carrying VC bridge) |
| BudgetCart                     | or-set rc     | BCart_ra_linearizable3_eq      | budgeted cart, spend derived from live instances; safety gated on causal canonicity (OQ 31.7)                                                                                             |
| Mergeable queue (Peepul)       | direct join   | queue_ra_linearizable3         | no rc exists (enqueue clique); the merge itself is the linearization witness                                                                                                              |
| RGA (embedded-chain)           | direct join   | embed_ra_linearizable3         | RA-lin at every honestly reachable configuration; canonical sorted state, fold-canonical (e_fold_canon); code-parametric (Part II)                                                        |
| RGA (sided, two-sided)         | direct join   | sided_embed_ra_linearizable3   | for every side assignment; the one-sided datatype is its all-R fragment (Part II)                                                                                                         |
| Peritext (embed kernel, fused) | instantiation | peritextEmbed_ra_linearizable3 | one RGA at $\alpha := \text{char} @ \text{boundary}$ ; deleting a character never re-formats another (renderIds_del; Parts II-III)                                                        |

Table 1: The Sal production catalogue. The flat data types conclude through the ONE generic conditioned metatheorem (§10–§12) at the identity instantiation ( $\approx =$ , their VC bundles feeding the flat collapse `FlatGeneric_Bridge.lean`, instance theorems in `MRDT_Instances/` (per-RDT)); the direct-join instances conclude through the per-configuration join hook under honest reachability (§13). The contract-tier column is each data type’s Join-Lemma discharge content (`MRDT_Instances/`, one directory per RDT); the historical flat corollaries over the raw system remain there as internal steps.

**shape** — commutation-closure suffices for set-shaped merges, counter-comparison merges need full causal closure. Reunifying the two routes was expected to be routine — “full closure only strengthens what a data type may assume.” Mechanization shows it is not (`JoinLemma3C: no_proper_back_block, no_proper_front_block`): the naive reunification — re-run the Join Lemma induction with full-closure hypotheses — is refuted, because no single-event (or block) peel preserves full closure. What survives is a *closure-indexed* contract in which the data type *declares* its closure strength; the soundness bridge is uniform across strengths, so no reunification into one strength is needed. In the mechanization the closure-indexed contract `JoinLemma3CC` has the single soundness bridge `ra_linearizable3_of_joinC` (store versions are fully causally closed, so the bridge applies for any  $\mathcal{C}$  that full closure implies), and all nine production discharges route through it unchanged (`AllMRDTs_viaContract`): eight at weak closure, the enable-wins flag at full.

## 9 The catalogue, discharged

Table 1 is the empirical heart of the metatheory: the VC set is not merely adequate in principle — the data types people actually ship satisfy it. Three lessons from the sweep:

**Tombstones are a metatheoretic trivializer.** The catalogue’s celebrated hard cases — the RGA sequence type and the Peritext rich-text type — land in the *easiest* tier. The reason is almost embarrassing: with tombstones, deletion *adds* a record; every state component is grow-only; all operations commute; the merge is a blind join. All of the RGA’s genuine difficulty lives in the *read-side* traversal (turning the node soup into a sequence), which RA-linearizability of the state never touches. By contrast, tombstone-free designs (which delete physically) fall outside the eight VCs for a reason worth stating precisely: their commutation VCs are themselves reachability-conditioned (two inserts commute only on well-formed states), and the update layer above, like the paper’s, quantifies over all states. The conditioned metatheory of §10 exists to host exactly this class, and the embedded-chain RGA of Part II is its canonical occupant, discharged through the factored route of §13.



**Commutation class and delta class are independent axes.** The multi-valued register is all-commuting, yet *not* in the unconditional tier: its merge is intersection-shaped,  $(l \cap a \cap b) \cup (a \setminus l) \cup (b \setminus l)$ , and fails the delta laws on infeasible tuples exactly as the OR-set does. The boundary between tiers is drawn by *how the merge uses its LCA*, not by whether operations commute. (The compensation: all-commuting makes every punctured causal past empty,  $B = \sigma_0$ , so the feasible discharge needs no trichotomy at all.)

**The eight VCs certify real code on its historical fault line.** The enable-wins flag’s repository history contains a known-broken sibling: a global-counter variant whose compositional VC (`inter_right_top`) fails on DAG-shaped executions, the very defect per-replica counters were introduced to fix. The mechanized  $\sigma$ -facts prove the production merge computes exactly union-liveness — *verified on precisely the corner where the sibling fails*. This is the methodology working as intended: the metatheory does not merely bless the catalogue, it explains *why* the fixed design is right.

## 10 The conditioned metatheorem

The eight VCs of §7 quantify over *all* states, and §9 showed that is right for flat data types and fatal for state-dependent ones: for a sequence datatype with physical deletion, two concurrent operations commute only on well-formed states at which both are applicable, and written over all states the commutation VC is simply false. The remedy is the general signature of §2: condition every VC on the state invariant `Inv` and the applicability predicate `app`, relax the witness fold of Definition 3.2 to the data type’s observational equivalence  $\approx$ , and prove RA-linearizability from the conditioned VCs once and for all. The developer’s burden is only to describe sensible operations; the metatheorem does the rest. This section and the next two state the conditioned VCs and the metatheorem with its pen-and-paper proof; §13 then presents the factored discharge route the production instances actually use.

## 11 The conditioned verification conditions

A datatype presents a signature  $\langle \text{State}, \sigma_0, \text{do}, \text{merge}, \text{Inv}, \text{app} \rangle$  where `Inv` is a predicate on states and `app` a predicate on (operation, state) pairs (Lean: `ConditionedMRDTSig, Framework/MRDTSig.lean`). It must supply:

**VC 1** (Discipline). *Inv  $\sigma_0$  holds; do preserves Inv on applicable operations; and generation is disciplined: every operation is app and fresh at its birth state, with globally unique, monotone ids. (This is the execution model, Lean: `ConditionedConfiguration, Framework/ConditionedExecutionModel.lean` — kernel-clean, axiom-free.)*

**VC 2** (Conditioned commutation — the swap witness). *For concurrent operations  $a, b$  and any state  $\sigma$  with Inv  $\sigma$  at which both are applicable in the relevant reorder sense,  $\text{do}(\text{do } \sigma a) b \approx \text{do}(\text{do } \sigma b) a$ .*

**VC 3** (Threadable reorder invariant). *An auxiliary invariant  $J$  on (state, pending-event) that (i) certifies applicability/the swap precondition of VC 2 at reorder states, (ii) holds at the enabling fold of each event, and (iii) is preserved by each reorder step. Only the enabled events — those whose causal past is folded — need be certified; that scope is exactly what a linearization repair ever transposes.*

**VC 4** (Merge–linearization bridge — the conditioned Join Lemma). *For a reachable LCA state  $l$  and branches  $a, b$  obtained from  $l$  by disjoint concurrent event lists,  $\text{merge } l a b \approx \text{fold } l \pi$  for a  $\text{lo}^E$ -respecting enumeration  $\pi$  of the branch events.*

For flat datatypes  $\text{Inv} = \text{app} = \text{true}$ , VC 2 degenerates to unconditioned commutation, VC 3 to  $J = \text{true}$ , and VC 4 to the ordinary Join Lemma — recovering the unconditioned methodology.

## 12 The metatheorem

**Theorem 12.1** (Soundness — conditioned VCs  $\Rightarrow$  RA-linearizability). *Any MRDT satisfying VCs 1–4 is RA-linearizable (Definition 3.2, read up to  $\approx$ ).*

The proof is generic — it never inspects a particular datatype — and factors through two lemmas, each proved over the abstract signature with  $\approx$  an arbitrary congruence for **do** and **merge**.

**Lemma 12.2** (Convergence). *VCs 1, 2, 3 imply that all  $\text{lo}^E$ -respecting admissible enumerations of a backward-closed reachable  $E$  fold from  $\sigma_0$  to  $\approx$ -equal states; hence  $\sigma^*(E)$  is well defined up to  $\approx$ .*

*Proof.* Any two  $\text{lo}^E$ -respecting enumerations of  $E$  are related by repeatedly transposing adjacent  $\text{lo}^E$ -incomparable events. Because  $\text{lo}^E$  is non-transitive, “respecting” is the elementwise **Pairwise** condition rather than sortedness, so this order-repair is more delicate than for a total order — it is exactly what the generic engine **eq\_convergence** carries out, and we appeal to it rather than re-derive it. Each transposition preserves the fold up to  $\approx$  by VC 2, provided the swap precondition holds at the transposition point; VC 3 supplies it along the whole enumeration (base at each event’s enabling fold, preserved by each step, scoped to the enabled events — the only ones the repair transposes). The engine consumes only that  $\approx$  is an equivalence, **do** is  $\approx$ -congruent, and a swap oracle of the shape VC 2+3 provides (Lean: **eq\_convergence**, **kernel-clean**).  $\square$

**Lemma 12.3** (Canonical adequacy). *VC 4 and Lemma 12.2 imply that every reachable version state is  $\approx \sigma^*$  of its delivered event set.*

*Proof.* Induction over the version DAG. Initial: the empty fold. Update node: appends one delivered operation to a canonical fold (definitional). Merge node with LCA  $l$  and branches  $a, b$ : by hypothesis  $a \approx \sigma^*(E_a)$  and  $b \approx \sigma^*(E_b)$ , and  $l \approx \sigma^*(E_l)$ . First rewrite  $a, b$  to literal canonical folds using  $\approx$ -congruence of **merge** (so VC 4, stated for branch folds, applies); then VC 4 gives  $\text{merge } l a b \approx \text{fold } l \pi$  for a  $\text{lo}^E$ -respecting  $\pi$  of the branch events  $E_a \cup E_b$ . Now  $E_l$  causally precedes those branch events (they were generated on states derived from  $l$ ), so a  $\text{lo}^E$ -respecting enumeration of  $E_l \cup E_a \cup E_b$  factors as (an  $\text{lo}^E$ -enumeration of  $E_l$ ) followed by  $\pi$ ; hence  $\text{fold } l \pi = \text{fold } (\sigma^*(E_l)) \pi \approx \sigma^*(E_l \cup E_a \cup E_b)$  by Lemma 12.2 (the engine is start-state generic). Backward closure keeps every enumeration admissible.  $\square$

*Proof of Theorem 12.1.* Immediate from Lemma 12.3: each version state  $\approx \sigma^*(E_v)$ , which is Definition 3.2. (Unconditioned template: **ra\_linearizable3\_of\_joinC** plus the closure-indexed adequacy bridge, already **kernel-clean** with nine instances; the conditioned metatheorem generalizes it by carrying **Inv/app** through the two lemmas.)  $\square$

**Corollary 12.4** (Strong eventual consistency). *Two versions with the same delivered set  $E$  have  $\approx$  states: both  $\approx \sigma^*(E)$  by Theorem 12.1, hence  $\approx$  each other.*

**Remark 12.5** (The flat instances). *The nine flat production MRDTs (counter, OR-set, MVR, ...) take `Inv = app = true`: VC 2 is their unconditioned commutation, VC 3 is `J = true`, and VC 4 is the ordinary Join Lemma — already discharged (`ra_linearizable3_of_joinC`). They inherit RA-linearizability from the same Theorem 12.1. The instances of §13 and Part II exercise the conditioned mechanisms proper.*

## 13 The factored discharge route: honest reachability and per-configuration joins

The conditioned instances in production do not each re-prove VCs 2 and 3 from scratch. What they share is an induction: thread a per-configuration honesty contract through reachability, and invoke a Join Lemma at each merge. This section presents that induction, extracted once, together with the instances that ride it: the mergeable queue (the flagship, whose conflict graph admits no `rc` at all), the bounded counter (conditioning for safety), and the FWW reservation register (payload arbitration). Part II’s embedded-chain RGA is discharged by exactly this route.

**The factorization: one metatheorem, three join species.** The common structure across the conditioned instances is a theorem rather than a convention. The developments all ran the same induction — thread a per-configuration honesty contract through reachability, invoke a Join Lemma at each merge — and that induction is extracted once (`Metatheory/HonestReach.lean`): `HonestReach D H` is LTS reachability where every step leaves a configuration satisfying the contract  $H$ , and the metatheorem (`ra_linearizable3_of_honest_reach`; axioms `propext`, `Quot.sound`) states that if every  $H$ -configuration admits the Join at its core (`JoinLemma3At`), every  $H$ -honestly reachable configuration is per-version RA-linearizable. Plain reachability is the degenerate contract  $H = \top$ , recovering the unconditional bridge; the queue’s and the bounded counter’s bespoke inductions are one-line corollaries of the extracted form.

What does *not* factorize is the join discharge, and that is the finding: the instances discharge `JoinLemma3At` by genuinely different mechanisms — *conditioned commutation* (operations commute on invariant-satisfying states, the framework’s fully general quotiented route), *flat commutation* (the bounded counter: the CD route, contract  $\top$ ), and *direct witness* (the queue: the merge is its own linearization certificate; Part II’s embedded-chain family discharges the same way). Conditioning thus splits as

$$\text{generic honest-reachability metatheorem} \times \text{per-datatype join discharge},$$

and the framework’s job at the boundary is to *receive* a join, not to produce one. A witness-classed generalization (`JoinLemma3AtW`, `Metatheory/WitnessClass.lean`, from the Shesha investigation of Part III) extends the hook to datatypes whose canonical states are unique only per witness-classed enumerations; and the instance layer is payload-generic where it should be (the embed instance of Part II takes its element type as a defaulted parameter, so the fused Peritext is a pure instantiation). One further regularity is worth recording: the production honesty contracts all have the shape “ $P$  holds of each event at the fold of its causal past” —

anchor liveness for the sequence family, slack for the counter, head-ness for the queue — with  $P$  the signature’s client-checkable `app` (or its accuracy analogue). That shape is extracted once (`GenHonest`, with `AppHonest` its `app` instance and `ra_linearizable3_of_genHonest_reach` the composite metatheorem; `Metatheory/GenHonest.lean`), and the counter’s and the queue’s contracts are re-derived as instantiations (`BCHonest_iff_genHonest`, `qHonest_of_genHonest`). A generic *safety* metatheorem completes the factorization; it is the subject of a paragraph below, and the counter’s counting argument indeed turned out to mark where the per-datatype content sits.

**The flagship direct-join instance: the mergeable queue — when no rc can exist.** The queue of the certified-MRDT line of work (Soundarapandian et al., PLDI’22, *Certified Mergeable Replicated Data Types*) is the catalogue’s first data type whose conflict graph is a *clique* rather than a star: two concurrent enqueues genuinely do not commute (queue order is arrival order), so VC 1’s iff forces an `rc`-edge between them, and any orientation of a self-conflicting class composes with itself into a length-2 path — every candidate assignment violates `no_rc_chain` (Open Question 31.6). The flat engine is structurally unavailable, and the instance goes through the framework’s per-configuration join hook instead (`JoinLemma3At`, `goodConfig3_merge_at`): prove the Join Lemma directly, at every reachable configuration.

The implementation is the functional queue with Peepul’s merge, verbatim: the state is a list of  $(tag, value)$  pairs, head first; `enq v` appends a fresh element tagged with the operation’s own timestamp; `deq t` removes the element tagged  $t$ , wherever it sits; and the three-way merge keeps the LCA’s elements that survive in both branches (in LCA order), then each branch’s new arrivals (in branch order). The client-facing API is unchanged — `dequeue()` at a replica reads its local head — but the *operation* records which element that call removed: `app` for `deq t` is “ $t$  is the head of the issuing replica’s state”. As with the sequence family, the generation discipline is the datatype’s natural client behaviour, captured rather than imposed.

The proof exhibits Peepul’s merge as its own certificate. The Join Lemma asks for a delivery-respecting enumeration of  $E_1 \cup E_2$  whose fold is `mergeL`( $\sigma_0, \sigma_1, \sigma_2$ ); the witness is the concatenation  $\rho_0 \cdot (\rho_1 \setminus E_0) \cdot (\rho_2 \setminus E_0)$  of the given enumerations — exactly the merge’s shape, no reordering. Three facts make the fold of that concatenation compute the merge, all consequences of the closure premise (branch event sets are backward-closed under  $\xrightarrow{\text{vis}}$  among non-commuting pairs), timestamp uniqueness, and an honest-history contract (`QHonest`: every dequeue names a tag whose enqueue is  $\xrightarrow{\text{vis}}$ -prior — the queue’s honest-delivery contract, discharged by the `app` head-check via `qHonest_of_applicable`): a fold from the empty queue is “the enqueues, in order, minus the dequeued tags” (the fold formula, which needs honesty — a dequeue preceding its enqueue would consume nothing); a delta’s tags are fresh with respect to the LCA (uniqueness); and a cross-branch dequeue is impossible — a `deq t` in one branch pulls its enqueue into that branch by closure, so if the enqueue of  $t$  is the *other* branch’s delta, it would lie in both branches, hence in the LCA, contradicting delta-ness. An LCA element then survives the union iff it survives in both branches, which is precisely the merge’s first clause.

The payoff is the specification. In the PLDI’22 development the queue was the hardest case exactly because its specification had to be written in a bespoke relational language over event partial orders, with an explicit clause recovering *which element is the head*; the merge was then verified against that ad-hoc spec. Here the specification is the same generic statement as every other instance in Table 1 — every version is the fold of a delivery-respecting linearization of its event set — and head-ness, FIFO order, and once-per-element consumption are *read off*

the fold rather than specified. (One semantic caveat, present equally in Peepul: two replicas that concurrently dequeue the same head both consume that one element — the merged state removes it once. RA-linearizability makes this visible instead of hiding it: both `deq t` events fold over the single `enq t`.) At  $\sim 870$  lines (`MRDT_Instances/MergeableQueue/`), the instance carries positional structure at a fraction of a sequence datatype’s cost.

**Conditioning for safety: the bounded counter.** Conditioning first entered the framework to deliver *convergence* for state-dependent datatypes. The bounded counter (per-replica escrow: grow-only tallies (`incs,decs`) per replica, per-slot group merge; mirror of `Sal/CRDTs/Bounded_Counter`) shows the same mechanisms delivering *safety* for a datatype whose convergence is flat: all its operations commute, the eight VCs discharge along the counter route, and the catalogue capstone (`BC_ra_linearizable3_eq`) is an identity instantiation. What the flat theory cannot even state is the property the datatype is named after. Its CRDT development is explicit: “the bound is enforced operationally by well-behaved clients” — a client refuses to issue Dec without slack — and that discipline lives outside the formal development. Conditioned, it moves inside: `Inv` is the per-replica escrow invariant ( $0 \leq \text{decs } r \leq \text{incs } r$ ), `app` is the client’s check (a Dec needs slack in the *issuing replica’s own* slots — positive, and checkable against the issuing replica’s state), and the contract on executions (`BCHonest`: every decrement was applicable at the fold of its causal past) is the counter’s honest-delivery contract. The theorem (`bc_version_inv`, kernel-clean): in every reachable configuration with honest history, *every version — heads, LCAs, everything the store registered — satisfies the invariant*; hence the counter’s value is non-negative everywhere (`bc_value_nonneg`). The proof is a counting argument riding the flat machinery: every version’s state is a fold of its event set (canonicity), fold slots *are* per-replica event counts (the group merge is inclusion–exclusion on counts, via the LCA lemma), and the vis-maximal decrement’s honesty at its own causal past — which lies inside the version by causal closure — bounds the decrement count by the increment count. At  $\sim 600$  lines, it is also a calibration point for which conditioning machinery is generic: the same invariant-at-generation shape, none of the positional structure.

**The generic safety metatheorem: causal witnesses, and an escrow route.** The safety half of conditioning also factorizes, but the obvious formulation is *false*, and the refutation comes from the flagship safety instance itself. One would like: if updates preserve `Inv` on applicable operations and the history is honest, then `Inv` holds at every version — proved by inducting along the version’s canonical enumeration. But canonicity constrains the enumeration only by `lo`, and for the bounded counter — all of whose operations commute — `lo` is *empty*: `[dec, inc]` is a perfectly canonical enumeration of an honest version, and its intermediate state violates the escrow invariant. Only *endpoints* of canonical folds are guaranteed; commuting causal predecessors may float behind the event that needs them. The repair is to restrict the witness class, not the induction: a *causal* witness additionally linearizes  $\xrightarrow{\text{vis}}$  (`CausalCanonical`), and its prefixes are exactly the vis-closed, future-free sets containing each event’s whole causal past. Over those, the metatheorem holds (`version_inv_of_causal_canonical`; axioms `propext`, `Quot.sound`): `Inv` at every version, from honesty (`HonestApp`: `app` at some causal fold of the event’s causal past — what the issuer’s own state witnesses) and one per-datatype obligation `SafetyStep` (applicability transfers from the causal past to any admissible prefix, fused with `Inv`-preservation). For datatypes whose operations all commute the causal witness is free (`causalCanonical_of_all_comm_rc_either`, a pointwise permutation argument). The

bounded counter discharges **SafetyStep** in three lines — extras in a prefix are other replicas’ events and cannot touch the issuer’s slots — and **bc\_version\_inv** is re-derived as a corollary, retiring the bespoke vis-maximal counting apparatus, which is thereby revealed as compensation for inducting over non-causal witnesses. A second, independent route generalizes that retired argument itself: for datatypes *measured* by affine observations (fold values = event-set counts), an *escrow metatheorem* (**escrow\_version\_inv**) yields the same conclusions with no causal witness and no **Inv**-preservation at all; **bc\_version\_inv** is derived a second time this way. The queue, by contrast, *refuses* **SafetyStep** — “*t* is the head” is not stable under a concurrent honest dequeue of the same head, and correctly so: the double-dequeue execution satisfies any sound obligation, and the queue’s stable residue is event-level (**QHonest**), consumed by convergence rather than by a state invariant. The full analysis, including two further refuted formulations, is the pen-and-paper memo `Development/GENERIC_SAFETY_PENPAPER.md` — written, per this project’s standing lesson, before any Lean. (A natural follow-up is a *conditioned step relation* — apply guarded by **app** at the head state — under which honesty becomes a theorem of the LTS rather than a contract; deferred.)

**A small fourth instance: the FWW reservation register — payload arbitration, and what honesty cannot buy.** A register claimable once (a seat, a username): sequentially the first claim wins; concurrently, the winner is the claim that is first in *Lamport* time. This is the positive complement to the metadata-free kill-test (**lww\_merge\_needs\_timestamps**): arbitration whose key is not in the state is impossible, and arbitration whose key *is* in the state is a semilattice triviality — the state is the minimum claim (**ts**, **rep**, *v*) under the lexicographic order (or unset), update and merge are min, and the entire eight-VC discharge is min-algebra (~300 lines, the smallest conditioned instance). The characterization theorem (**fww\_version\_min**) is honesty-free — semilattice folds are enumeration-independent — and says: at every version of every reachable configuration the register holds exactly the minimum-timestamp claim of its event set, unset exactly on the empty set. Since timestamps respect causality, a causally later claim never displaces an installed one; the min rule arbitrates only among *concurrent* claimants, and it must, on pain of divergence. The instructive part is the generation discipline: **app** for a claim is “the register is unset in my causal past”, and *deliberately no theorem consumes it* — two honest concurrent claimants both see unset, both claim, and each locally wins until a merge disabuses one. “Unset” is the minimal example of an applicability check that is *not stable under concurrent honest extension* (contrast the escrow counter’s own-slot slack, which is — precisely the boundary the safety metatheorem’s **SafetyStep** draws). A merge-based register is a reservation, confirmed only at causal stability; it is never a mutex.

## Part II

# The embedded-chain family

The embedded-chain RGA (replicated growable array) is a replicated list that keeps no tombstones, no ledger, and no event log: its entire state is the live nodes. It is a sequence MRDT in Part I’s sense: state-based, reconciled by a three-way merge against the branches’ lowest common ancestor version. A deleted element survives only as  $\Theta(\log \delta)$  bits inside its descendants’ sort keys,  $\delta$  being the Lamport-timestamp gap between the element and its own insertion anchor (1 under continuation typing); that is the log of the event gap the insertion crossed, a race or a

cursor jump, paid once, in space. Concurrent inserts provably cannot tie, so the merge contains no repair and never decides an order; the state is a canonical function of the event set, and convergence is literal equality. Observationally the datatype *is* the published RGA with the tombstone set compressed into coordinate entropy: a kernel-checked read-equivalence theorem, with machine lockstep at every step as its tested form. All of this follows from one idea: an element’s sort key is its *birth chain*, the timestamps from the root of the birth tree down to the element, dead ancestors included. The chain is written at birth, never edited, and carried in an order-preserving self-delimiting code. Separating the datatype from its encoding makes the metatheory nearly definitional: correctness is three one-line facts about chains, plus a single faithfulness theorem about the code. The design is machine-validated (the L1–L25 anomaly battery, except L19, RGA’s own, discharged by the sided generalization of §18; 420+ randomized version-DAG executions; 120/120 lockstep), and the central theorem, RA-linearizability at every honestly reachable configuration, is a kernel-checked Lean theorem (§21) discharged by the direct-join route of §13. (Working names in the validation artifact: `embed-tree` for the unary code, `embed-code` for the delta-coded one.)

## 14 The datatype: sort keys are birth chains

Timestamps  $t \in \mathbb{N}^+$  are Lamport stamps, globally unique, and double as element identities. Causality gives: the anchor of every insert carries a smaller timestamp than the insert itself.

**The birth tree.** Every insert names an anchor. The event set therefore induces a tree: the root is the front sentinel 0; the *birth parent* of each element is its anchor. The tree is append-only and never edited: an element’s place in it is fixed at birth.

**Birth chains.** The *birth chain* of  $u$  is the sequence of timestamps from the root down to  $u$  (sentinel elided):  $\text{chain}(u) = \text{chain}(a) \cdot u$  when  $u$  was inserted after anchor  $a$ , and  $\text{chain}(u) = \langle u \rangle$  at the front. Dead ancestors are *not* elided: their timestamps remain in their descendants’ chains. The chain is the sort key, and everything in this section is stated in terms of it; nothing here depends on how chains are stored (§15).

**Type.** A state is a finite map over *live nodes only*:

$$\sigma : u \mapsto (\text{chain}(u), \text{char}_u).$$

**init.**  $\sigma_0 = \emptyset$ .

**do (the operations).** Two operations, with  $\text{rc} = \text{Either}$  everywhere (§16):

- $\text{ins}(x \leftarrow a, c, \pi)$ : insert element  $x$  (its timestamp) with character  $c$  after anchor  $a$  ( $a = 0$  for the front). The payload  $\pi = \text{chain}(a)$  is the anchor’s birth chain, carried *for the proof alone* (§16). Application on a state where  $a$  is live (every reachable application, by honesty):

$$\sigma[x] := (\text{chain}_\sigma(a) \cdot x, c), \quad \pi \text{ unread.}$$

No head-tracking, no reading of siblings.



- **del( $d$ ):** remove  $d$ 's record. Nothing else changes: survivors' chains are untouched; in particular,  $d$ 's descendants still carry  $d$ 's timestamp in their sort keys. If  $d$  is absent, no-op (concurrent double delete).

**read.** Sort survivors by *chain-lex*: compare chains timestamp-by-timestamp with *larger first*; a proper prefix precedes its extensions (a live ancestor displays immediately before its descendants). Among same-parent siblings this is exactly RGA's newest-first rule; equivalently, the canonical order is the RGA traversal of the birth tree restricted to survivors.

**merge.**  $\text{merge}(L, A, B)$ , where  $L$  is the LCA (the branches' lowest common ancestor version, the merge context the MRDT model stores and supplies). The merge is *survival* (OR-set): live iff in all three, or born in exactly one branch. There is no second step at this layer: survivors' chains come with their insert events, so every input that holds a survivor agrees on its chain. The merge never compares timestamps and never decides an order.

**Inv / applicable / honesty.** The honesty contract is the standard RGA generation discipline: an insert is *generated* at a replica where its anchor is live (you insert after a character you can see), and *ins/del* are *applied* on states where they are applicable (anchor live; delete of a live or already-dead node). On every reachable state, application never consults  $\pi$ . Canonicity (Theorem 14.1) doubles as Inv; at the representation layer, Theorem 15.1(iii) is its stored form.

$\approx$ . Equality. The state is a canonical function of the event set (Theorem 14.1), so no quotient is needed: convergence is literal state equality.

**The metatheory, which is nearly definitional.** At this layer the design's theorems all but evaporate; we record them because they are the specification the representation must meet (§15).

**Theorem 14.1** (chain-level metatheory). *In every reachable state:*

- (i) *Canonicity:*  $\sigma = \{ u \mapsto (\text{chain}(u), \text{char}_u) \}$  over survivors  $u$ ; the state is a function of the event set.
- (ii) *Birth constancy:*  $\text{chain}(u)$  is fixed at  $u$ 's insert and never edited by any later transition.
- (iii) *Totality (no ties):* *chain-lex* is a total order on every state's survivors.

*Proof.* (i) Survivorship is OR-set, hence a function of the event set, and a survivor's chain is determined by its insert event alone: by induction, *ins* writes  $\text{chain}_\sigma(a) \cdot x = \text{chain}(a) \cdot x$  (the anchor is live at application, by honesty, and its stored chain is canonical), while *del* and the merge only delete or select records, never rewrite them. (ii) is (i) read off per node: no transition writes to an existing record. (iii) Distinct elements' chains are distinct (they end in distinct timestamps); if neither is a prefix of the other, the first difference compares two distinct timestamps in  $\mathbb{N}$ , and uniqueness of Lamport stamps is the no-tie property; prefix pairs are ancestor/descendant, ordered by the prefix rule.  $\square$

**The one-line separation, and the conservation law.** The natural first attempt at tombstone-freedom is the *rehoming* RGA: keep flat (element, anchor) records, and let a delete splice its node out, re-parenting the orphans to the nearest live ancestor (the *climb*). We built and verified that design before this one (it is RA-linearizable, in the same Lean framework), and it fails the L1 delete-reorder litmus (the  $[b, a, c] \rightarrow [c, b]$  anomaly, Figure 4): the climb re-sorts a rehomed node by its own timestamp, so *its sort key is the birth chain truncated at each rehoming*. This design keeps the chain whole. That is the entire difference between the two datatypes, and it is the crisp form of the conservation law that our design sweep kept re-imposing: *the sort key must retain dead ancestors' timestamps*. Every design that forgets them has a named machine-checked countermodel (§20). What is negotiable is only how many bits the retained timestamps cost; that is the subject of §15, and the answer is: the log of each element's anchor gap, and nothing else.

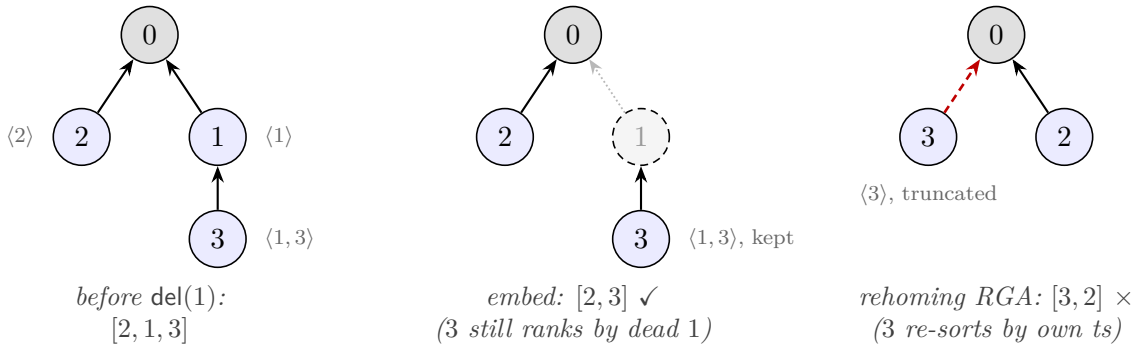


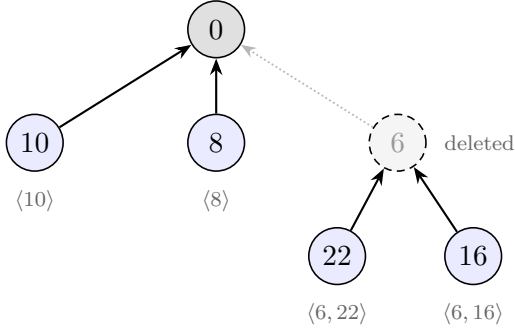
Figure 4: The one-line separation on the L1 delete-reorder litmus: insert 1 and 2 at the front, 3 after 1, delete 1. The rehoming RGA's climb (red) re-parents 3 and re-sorts it by its *own* timestamp: the truncated key  $\langle 3 \rangle$  beats  $\langle 2 \rangle$ , and 3 jumps over 2. The embedded chain  $\langle 1, 3 \rangle$  keeps ranking 3 by its dead parent's timestamp:  $[2, 3]$ , the delete-order-preserving read. Arrows point from a node to its anchor; dashed grey = dead.

**Worked example (Figure 5).** 6 and 10 concurrently insert at the front: chains  $\langle 6 \rangle$  and  $\langle 10 \rangle$ ; display  $[10, 6]$ , no arbitration, at every replica and merge order. Type 22, 16 under 6: chains  $\langle 6, 22 \rangle$ ,  $\langle 6, 16 \rangle$ . Delete 6: the records of 22 and 16 are untouched and still carry 6's timestamp. A third party concurrently inserts 8 at the front:  $\langle 8 \rangle$  sorts below  $\langle 10 \rangle$  and above  $\langle 6, 22 \rangle$ , so the display is  $[10, 8, 22, 16]$  whether 8 arrives before or after 6's death (machine-checked as the *credential countermodel*, the shape that kills every timestamp-oblivious variant, §17).

## 15 The representation: chains as entropy-coded coordinates

Everything above is the datatype; everything below is bits. A naive carrier would store a full timestamp at every chain level (each  $\sim \log(\text{document age})$  bits, even for purely sequential typing) and compare whole chains at every read. This section stores the chains compactly without disturbing their order.

**Vocabulary.** A *codeword* is the bit string  $C(\delta)$  encoding one chain level: one element's timestamp, stored as its differential  $\delta$  to its birth anchor (step 1 below). A *coordinate* is the bit



*display = chain-lex, larger first:*

10 8 22 16

*<8> vs <6, 22> compares 8 against the dead 6: the orphans stay in 6's slot, and 8 cannot land between them, whether or not 6's death has arrived.*

Figure 5: The credential countermodel as a birth tree. 6 and 10 race at the front; 22, 16 are typed under 6; 6 dies; 8 arrives at the front concurrently. Survivors sort by whole birth chains, so the read is  $[10, 8, 22, 16]$  at every replica and merge order. Every timestamp-oblivious variant flips this shape (§20).

string encoding a whole chain: the concatenation, root to element, of one block per level (the level's codeword plus three framing bits fixed by the mint construction below), with nothing else marking the boundaries. We write  $|s|$  for the length in bits of a bit string  $s$ . Read as a binary fraction, a coordinate names a dyadic interval in  $(0, 1)$ ; codeword concatenation becomes interval nesting, and the interval is the  $\alpha(u)$  of Theorem 15.1. Bit string and interval are two views of one object; we use whichever is clearer.

**What the coding is for.** Two jobs, only one of which is about correctness. *Job 1: cost.* The datatype layer already forced sort keys to retain dead ancestors' timestamps (the conservation law, §14); the only remaining question is how many bits that retention costs. Without entropy coding, “tombstone-free” would be bookkeeping: a full timestamp per level grows with document age however calm the history. The *unary control* (**embed-tree**: the identical datatype with a unary code in the mint, codeword length linear in the value, kept as the experiment's cost-control arm; step 2 and the entropy-law table) makes the gap measurable: 61 KB against the coded design's 0.5 KB on the table's 1000-element chain scenario. The coding is what makes the headline claim true: the dead survive as the entropy of the anchor gaps they were born across, and not more. *Job 2: faithfulness.* The chains must be stored so that plain lexicographic bit comparison reproduces chain-lex exactly, in every reachable state, forever. This is the single obligation the representation owes §14 (Theorem 15.1). Correctness lives upstairs; this layer is an implementation that must not lie. Job 2 needs no entropy optimality at all: it needs exactly two structural properties of the per-level code, stated next.

**The contract.** The stored form must support four operations: *mint* (insert: produce the new codeword from the two timestamps alone, coordination-free, reading no siblings); *compare* (read: plain lexicographic comparison, with no tie rule); *fold* (delete: splice a dead node out by concatenating its block into each child's stored string, exactly); and *refold* (merge: recompute survivors' coordinates by the same concatenation). Each guarantee is bought by a specific property of  $C$ :

- **Monotone** ( $\delta < \delta' \Rightarrow C(\delta) <_{\text{lex}} C(\delta')$ ) buys correct *compare*. A chain-lex first difference always compares two same-anchor entries, where timestamp order and  $\delta$  order coincide

(step 1); bitwise comparison of coordinates therefore equals timestamp comparison at the first differing level precisely when  $C$  is monotone.

- **Prefix-free** (no codeword is a prefix of another) buys three things at once. *Alignment*: in a comparison of two coordinates, the first differing bit falls inside the first differing level’s codewords, so levels cannot be confused across boundaries and no delimiters are needed; this is also what keeps *fold* and *refold* exact splices. *Geometry*: by the Kraft correspondence, prefix-free codewords occupy pairwise-disjoint dyadic cells, which is Theorem 15.1(i) and the reason a concurrent front-insert can never land inside a dead sibling’s span (Figure 7). *No ties*: cell disjointness plus Lamport uniqueness means sibling coordinates never tie, so the merge never decides an order and the read needs no tiebreak.
- **Universal, with a cheap floor** buys Job 1:  $C$  is defined for every  $\delta \geq 1$ ,  $|C(1)| = O(1)$  so continuation typing costs a constant per level, and  $|C(\delta)| = \Theta(\log \delta)$  so a level pays the log of its anchor gap, whether the gap is a race or a cursor jump (step 1), and nothing else.

Monotone plus prefix-free is all of Job 2: any such code satisfies Theorem 15.1, which is why the Lean capstone is parametric in the code (§21) and why the unary control proves the same theorems at absurd cost. Entropy coding enters only in *which* monotone prefix-free code is chosen; the choice sets the constant in the bits-per-level bill and nothing in the correctness story. One requirement cuts across all four operations: the arithmetic is exact, never rounded (see the binary64 refutation below).

**Step 1: differential coding.** By causality  $\delta = x - a \geq 1$ . A chain-lex first difference always compares two entries with the *same* anchor (the shared prefix), and for a fixed anchor, ordering by  $x$  and by  $\delta$  coincide, so each chain level may store the differential instead of the timestamp. Call  $\delta$  the *anchor gap*: the events elapsed between the anchor’s mint and the insert’s. Continuation typing (each character anchored at its predecessor) has  $\delta = 1$  at every level regardless of how long the document has lived. Two things widen the gap, and they pay the same bill: a genuine race with timestamp gap  $g$  costs  $\log_2 g$  bits, and so does a *cursor jump*, the first character typed at an old anchor; its successors anchor at fresh characters and are back to  $\delta = 1$ . The differential is an encoding choice, not semantics: the mathematical object remains the timestamp.

**Step 2: entropy coding.** For  $\delta \geq 1$  let  $L$  be the bit-length of  $\delta$  and

$$C(\delta) = \underbrace{1 \cdots 1}_{L-1} 0 \text{ } (\delta \text{ with its leading bit removed}) \quad (|C(\delta)| = 2L - 1),$$

an order-preserving prefix-free code: exactly the contract. The length accounting: prefix-freeness makes each codeword self-delimiting, and a self-delimiting code over unbounded  $\delta$  must spend bits twice, once on the value and once announcing its own length.  $C$  pays each bill at  $\sim \log_2 \delta$ . The header  $1^{L-1}0$  is a unary announcement “ $L-1$  payload bits follow” ( $L$  bits); the payload is  $\delta$ ’s trailing  $L-1$  bits, since the leading bit of an  $L$ -bit number is always 1: once the header has said  $L$  it carries no information and is dropped. Hence

$$|C(\delta)| = \underbrace{(L-1) + 1}_{\text{header}} + \underbrace{(L-1)}_{\text{payload}} = 2L - 1 = 2\lfloor \log_2 \delta \rfloor + 1,$$

e.g.  $C(1) = 0$ ,  $C(2) = 100$ ,  $C(3) = 101$ ,  $C(5) = 11001$ . This is Elias  $\gamma$  with its header flipped:  $\gamma$  announces the length in *zeros*, which is prefix-free but order-reversing across length classes: at the first divergence the longer, hence larger, number shows a header 0 against the shorter's leading payload bit 1. Writing the header in ones makes that same position decide in favor of the longer code, so lexicographic order on codewords is numeric order on  $\delta$ . The  $\gamma$ -flip is the building block, not the last word: only the *value* bill is forced at  $\log_2 \delta$ , and the *length* bill can itself be entropy-coded. Applying  $C$  to the length field (then the payload) gives the order-preserving flip of **Elias**  $\delta$ , prefix-free and monotone by the same across-length-class argument, at  $\log_2 \delta + O(\log \log \delta)$  per level; *this is the canonical mint*. Any order-preserving prefix-free code works here, and the artifact exists in three proved encodings of one datatype (**embed-tree** unary, **embed-code** the  $\gamma$ -flip, **eliasDeltaCode** the  $\delta$ -flip); the capstone theorem is inherited at each verbatim, with zero new proof content (§21): the code-parametricity claim, machine-validated. Two honest footnotes: per level the  $\delta$ -flip wins only from  $\delta \geq 32$  and loses at  $\delta \in \{2, 3\} \cup [8, 15]$  (kernel-checked length comparisons), and measured editing tails are thin, so on real traces the two flips are a wash; the dominance shows in race-heavy shapes (the entropy-law table below) and in the asymptotics.

**The mint.** Writing  $b = 1 \cdot C(\delta)$  (a leading 1 keeps coordinates positive) and  $w = 2^{-|b|-2}$ , the mint is the second quarter of  $b$ 's dyadic cell:

$$M(\delta) = (0.b + w, 0.b + 2w) \subset [0.b, 0.b + 4w).$$

The quarter placement leaves a gap below and headroom above inside the cell, and the open space  $(0.b + 2w, 0.b + 4w)$  is never minted: it exists so that the interval has room to *contain* the node's own subtree without touching the next cell. This also pins down the per-level *block* promised in the vocabulary:  $0.b + w$  written as a string is  $b01$ , so the block is  $1C(\delta)01$ , the codeword in its three framing bits (the leading 1 for positivity, the 01 that selects the second quarter). The block is what a node stores; read as a binary fraction it is the lower endpoint of  $M(\delta)$ , and flipping the trailing 01 to 10 gives the upper. Worked out for small deltas:

| $\delta$ | $C(\delta)$   | block = $1C(\delta)01$ | $M(\delta)$                | bits/level |
|----------|---------------|------------------------|----------------------------|------------|
| 1        | 0             | 1001                   | (9/16, 10/16)              | 4          |
| 2        | 100           | 110001                 | (49/64, 50/64)             | 6          |
| 3        | 101           | 110101                 | (53/64, 54/64)             | 6          |
| 4        | 11000         | 11100001               | (225/256, 226/256)         | 8          |
| 5        | 11001         | 11100101               | (229/256, 230/256)         | 8          |
| 10       | 1110010       | 1111001001             | (969/1024, 970/1024)       | 10         |
| 100      | 1111110100100 | 1111111010010001       | (65169/65536, 65170/65536) | 16         |

Three things to read off. The endpoints are consecutive dyadics at scale  $2^{-(|b|+2)}$  (numerators  $n$  and  $n+1$ ): the mint is one tick wide at its own scale, with the tick below it (the gap) and the two ticks above it (the subtree headroom) never minted. The rows are strictly increasing as fractions: monotonicity made visible. And the bit count is  $2L + 2$  for an  $L$ -bit  $\delta$ : four bits at  $\delta = 1$  (the sequential-typing constant of the entropy law), sixteen at  $\delta = 100$ .

**The stored state.** The carrier stores each live node parent-relatively:

$$\sigma : t \mapsto (\text{par} \in \mathbb{N}, \text{ s} \in \{0, 1\}^*, \text{ char}),$$

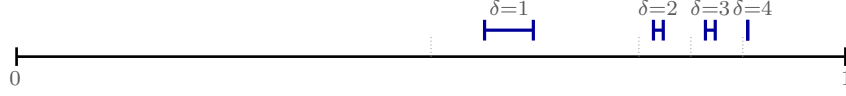


Figure 6: Mints  $M(\delta)$  for  $\delta = 1..4$  inside the parent's unit interval (cell boundaries dotted). Larger deltas sit strictly higher; distinct deltas are disjoint with gaps; each mint is a quarter of its cell, leaving room for the node's own subtree. Drawn for the  $\gamma$ -flip  $C$ ; the construction reads verbatim over any order-preserving prefix-free code, in particular the canonical Elias- $\delta$  mint (step 2).

where  $s$  is a block ( $1C(\delta)01$  at birth; after folds, below, a concatenation of blocks) *relative to the parent*, and  $\text{par} = 0$  denotes the root sentinel (which owns  $(0, 1)$  and is never stored). The record holds no interval; the interval is the string's decoding,

$$(lo, hi) = (0.s, 0.s + 2^{-|s|}) \subset (0, 1),$$

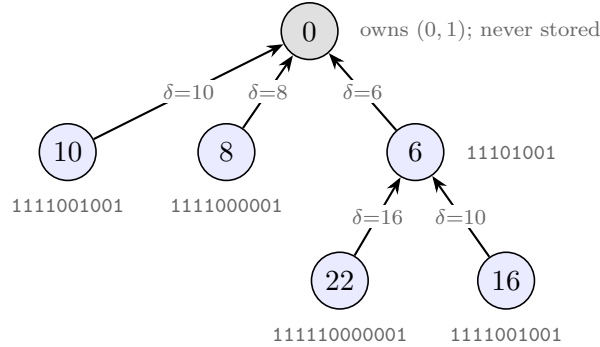
consecutive dyadics at  $s$ 's own scale, one block at a time exactly as in the mint table. The maintenance below and Theorem 15.1 are stated on the intervals, and every operation they perform on an interval is a string operation on  $s$ : composition is concatenation, comparison is lexicographic. (The validation model stores the decoded pair directly, as exact fractions.) A node's *absolute* interval is the affine composition of the intervals along its parent chain. For a node  $v$  with mint  $M(\delta_v) = (lo_v, lo_v + w_v)$ , let

$$\varphi_v : (0, 1) \rightarrow M(\delta_v), \quad \varphi_v(x) = lo_v + w_v \cdot x,$$

the unique increasing affine bijection of the unit interval onto  $v$ 's mint; it maps a subinterval  $(a, b)$  to  $(lo_v + w_v a, lo_v + w_v b)$ , and on strings it prefixes  $v$ 's block:  $\varphi_v(0.y) = 0.(s_v y)$ . Write

$$\alpha(u) = \varphi_{c_1} \circ \cdots \circ \varphi_{c_k}(M(\delta_u))$$

for the composition along  $u$ 's birth chain  $c_1 \cdots c_k u$  ( $\delta_v$  is  $v$ 's delta to its birth parent):  $\alpha(u)$  is the *encoding of*  $\text{chain}(u)$ , and at the string level it decodes the concatenation of the blocks along the chain. Worked on the tree of Figure 5, redrawn before  $\text{del}(6)$  with the delta (child minus anchor) on each edge and the stored block under each node:



Writing  $(n, n+1)/2^k$  for the interval  $(n/2^k, (n+1)/2^k)$  and  $\approx$  for its lower endpoint:

| node | $\delta$ | stores (par, $s$ ) | decode of $s$         | $\alpha$ (absolute)       | $\approx$ |
|------|----------|--------------------|-----------------------|---------------------------|-----------|
| 10   | 10       | (0, 1111001001)    | $(969, 970)/2^{10}$   | $(969, 970)/2^{10}$       | 0.94629   |
| 8    | 8        | (0, 1111000001)    | $(961, 962)/2^{10}$   | $(961, 962)/2^{10}$       | 0.93848   |
| 6    | 6        | (0, 11101001)      | $(233, 234)/2^8$      | $(233, 234)/2^8$          | 0.91016   |
| 22   | 16       | (6, 111110000001)  | $(3969, 3970)/2^{12}$ | $(958337, 958338)/2^{20}$ | 0.91394   |
| 16   | 10       | (6, 1111001001)    | $(969, 970)/2^{10}$   | $(239561, 239562)/2^{18}$ | 0.91385   |

The root's children store their absolute intervals already (par = 0). The grandchildren compose: 22's absolute string is its parent's block followed by its own, 1110100111110000001 (20 bits, hence the  $2^{20}$ ), and likewise 16's (18 bits). Both absolute intervals sit inside  $M(6) = (0.91016, 0.91406)$  while 8's sits above it, so the descending- $\alpha$  read is [10, 8, 22, 16] and 8 cannot land between the orphans. Nodes 16 and 10 store the same string, both being  $\delta = 10$  births; records are parent-relative, so this is no collision. After  $\text{del}(6)$  the fold rewrites the two records to (par 0, the concatenated  $s$ ) and the  $\alpha$  column is unchanged bit for bit. Figure 7 draws these intervals (widths schematic).

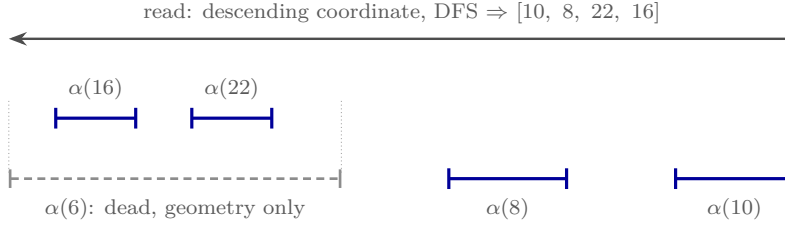


Figure 7: Figure 5 downstairs (widths schematic; true mints are the dyadic cells of Figure 6). Deleting 6 removes its record, but  $\alpha(16)$  and  $\alpha(22)$  are compositions *through*  $M(6)$ : the dead node survives as the dashed container, the tombstone compressed into geometry. 8's interval can never land inside  $\alpha(6)$ 's span: distinct deltas occupy disjoint cells (Theorem 15.1(i)), which is the credential verdict, now a fact about intervals.

**Maintenance.** How the stored form tracks the datatype:

- $\text{ins}(x \leftarrow a)$  mints  $M(x - a)$  relative to  $a$ : the new record depends only on the two timestamps.
- $\text{del}(d)$  *isometrically folds*  $d$  into its children: each child  $c$  is re-parented to  $d$ 's parent and  $d$ 's record is substituted into  $c$ 's. On the stored strings this is concatenation,  $c.s := d.s c.s$ ; read as intervals,  $c.\text{frac} := d.\text{frac} \circ c.\text{frac}$ , where  $I \circ J$  is the image of  $J$  under the increasing affine bijection of  $(0, 1)$  onto  $I$  (the  $\varphi$  of the stored state, with  $I$  in place of a mint): for  $I = (lo, lo + w)$ ,

$$I \circ J = (lo + w \cdot lo_J, lo + w \cdot hi_J).$$

The substitution is exact. Upstairs, deletion changes nobody's sort key (§14); the fold is the parent-relative storage scheme *re-normalizing* while every absolute interval stays fixed (Figure 8): representation maintenance, not datatype semantics. On the running example,  $\text{del}(6)$  removes 6's record and substitutes it into both children: 22's record



(6, 111110000001) becomes (0, 11101001 111110000001), whose decode is

$$\begin{aligned} (233, 234)/2^8 \circ (3969, 3970)/2^{12} &= (233 \cdot 2^{12} + 3969, 233 \cdot 2^{12} + 3970)/2^{20} \\ &= (958337, 958338)/2^{20}, \end{aligned}$$

and likewise 16's becomes (0, 11101001 1111001001), decoding  $(239561, 239562)/2^{18}$ . Both are exactly the  $\alpha$  column of the worked table: the absolute intervals did not move, only the parent-relative bookkeeping did.

- The merge *refolds*: for each survivor, compute its absolute interval by folding its record to the root inside the input that holds it (a record's stored parent is always live in its own input, so the chain stays in one input; by Theorem 15.1(iii) the choice of input cannot matter, asserted at runtime and never fired); re-parent to the nearest *surviving* ancestor; re-relativize.
- The read is depth-first from the root, children in descending order of *lo*. No tie rule is needed (Corollary 15.2).

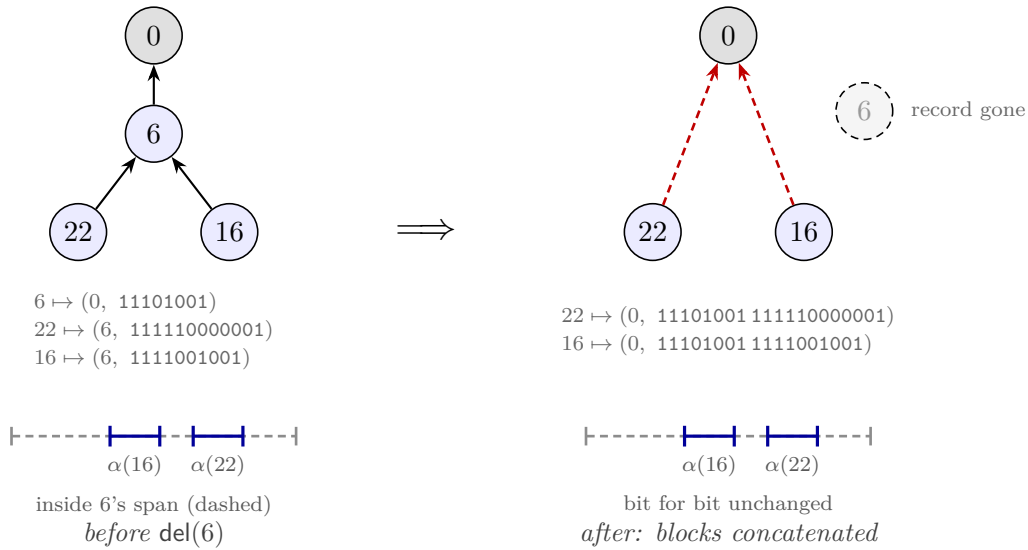


Figure 8: The isometric fold on the running example. `del(6)` deletes 6's record and concatenates its block 11101001 onto the front of each child's, re-parenting both to the root (red: the re-parenting, a storage move only). The stored states change: 22 and 16 now carry two-block strings and par 0. The absolute intervals  $\alpha(22) = (958337, 958338)/2^{20}$  and  $\alpha(16) = (239561, 239562)/2^{18}$ , and with them the display, are bit-for-bit unchanged: upstairs, the sort keys still read  $\langle 6, 22 \rangle$  and  $\langle 6, 16 \rangle$ . The dashed span is 6's old interval, surviving as pure geometry.

**Why the codeword must announce its own end.** Before the theorem, the countermodel that motivates it. A single codeword in a field of known extent needs no header, and the length looks recoverable from the string itself; plain binary shows what breaks, once per purchase in the contract. *Geometry:*  $B(2) = 10$  prefixes  $B(5) = 101$ , so  $B(5)$ 's cell sits inside  $B(2)$ 's;

containment means ancestry, so the  $\delta=5$  sibling lands amid the  $\delta=2$  sibling's descendants and non-interleaving dies with everything alive. *Parsing*: once a fold erases the structural boundary between blocks, 101100... reads as  $B(2)$  then 11... or as  $B(5)$  then 00..., and nothing outside the string can say. *Ties*: a binary fraction cannot see trailing zeros, so 10 and 100 both denote  $\frac{1}{2}$ ; distinct siblings mint touching intervals, and the id tiebreak wakes into an arbiter that orders by id, not chain-lex. Nor can the header bits be dodged: prefix-freedom demands  $\sum_{\delta} 2^{-\ell(\delta)} \leq 1$ , and plain binary puts  $2^{L-1}$  values at every length  $L$ , so every prefix-free code pays a length announcement of order  $\log L$  somewhere (out-of-band boundary tables cost the same bits and forfeit the string as a self-contained key; database key encodings inline it for the same reason). And the fold names where the boundary went: the tree node that carried it structurally is the deleted tombstone, whose ordering verdict becomes the payload and whose boundary becomes the header. “Tombstone compressed into coordinate entropy” is literal at the bit level.

**The one obligation.** The four theorems of the earlier drafts collapse into a single statement about the encoding.

**Theorem 15.1** (encoding faithfulness). *(i) Order-embedding at one level: for  $\delta \neq \delta'$ ,  $M(\delta)$  and  $M(\delta')$  are disjoint with a gap;  $\delta < \delta' \Rightarrow M(\delta)$  lies entirely below  $M(\delta')$ .*

*(ii) Homomorphism:  $\alpha$  realizes chain concatenation as affine composition and embeds chain-lex in the interval order: for distinct  $u, v$ , if  $\text{chain}(u)$  is a proper prefix of  $\text{chain}(v)$  then  $\alpha(v) \subset \alpha(u)$  (and, if both are live, the two are never read-time siblings); otherwise  $\alpha(u) \cap \alpha(v) = \emptyset$ , with the chain-lex-greater element's interval strictly higher.*

*(iii) Exact maintenance: in every reachable state, every live node's stored records compose to exactly  $\alpha(u)$ .*

*Proof.* (i)  $C$  is prefix-free: codes of lengths  $2L-1 < 2M-1$  differ at position  $L$  (the shorter's header terminator 0 against the longer's header 1); equal-length codes differ in the payload. Prefix-free codes have pairwise-disjoint dyadic cells.  $C$  is monotone as a binary fraction: within a length class the payload is  $\delta$ 's low bits in order; across classes the position- $L$  bit decides in favor of the longer code. Disjoint cells ordered by their codes, and mints interior quarters of their cells, give both claims. (ii) If neither chain is a prefix of the other, they first differ on distinct deltas relative to the *same* birth ancestor (§15, step 1); (i) separates them there, and affine images of disjoint intervals are disjoint and order-preserved (each  $\varphi$  is increasing). If  $\text{chain}(u)$  prefixes  $\text{chain}(v)$ , then  $\alpha(v) \subseteq \varphi_{c_1} \circ \dots \circ \varphi_{c_k}(\varphi_u((0, 1))) = \alpha(u)$  by construction; and if both are live,  $v$ 's nearest live ancestor is  $u$  or deeper, so they never share a current parent, and sibling groups are pairwise disjoint. (iii) By induction over transitions. *ins* establishes it by definition. *del*'s fold replaces (par, frac) pairs by their exact composition, leaving  $\alpha$  unchanged. The merge recomputes precisely  $\alpha(u)$  (folding a record chain to the root) and re-relativizes against another  $\alpha$ ; ties cannot exist by (i) and (ii), so no step of any transition ever rewrites an interval by anything but exact composition.  $\square$

**Corollary 15.2** (display  $\equiv$  chain-lex; no tie rule). *The read (DFS by descending lo) enumerates survivors in exactly chain-lex order: per level, larger delta first, so among same-parent siblings, exactly newest first; survivors rank by their own timestamps, folded heirs by their dead founder's. Sibling lo-values never tie; the read's id tiebreak is dead code. Machine checks: pairwise sibling*

*disjointness asserted at every read of 320 randomized executions, all clean; over 19,531 multi-member sibling groups, own-timestamp order violated 3,481 times (exactly the post-fold heir groups; forcing own-ts there is the refuted dissolution behavior), **chain-lex violated 0 times**.*

**The entropy law.** A coordinate is the concatenation of the codes along the birth chain:  $\Theta(\sum_i \log \delta_i)$  bits, the entropy of the chain. Continuation typing ( $\delta = 1$  at every level) costs  $\sim 4$  bits per level under every code here; a level minted across an event gap  $g$ , by a race or by the first character after a cursor jump, costs  $\log_2 g + O(\log \log g)$  bits under the canonical Elias- $\delta$  mint ( $2 \log_2 g + 1$  under the  $\gamma$ -flip). *The state pays for the magnitude of edit non-locality, concurrent or not, and for nothing else.* Measured (1000 elements; the numbers are *order metadata alone*, survivors’ coordinate bits read off the dyadic denominators; characters are excluded, since data costs are identical across all designs):

| scenario                             | quarter carve<br>(refuted) | unary mint                   | $\gamma$ -flip mint         | Elias- $\delta$ mint                  |
|--------------------------------------|----------------------------|------------------------------|-----------------------------|---------------------------------------|
| chain then delete-all ( $\delta=1$ ) | $2^{2001}$                 | $2^{502501}$ ( $\sim 61$ KB) | $2^{4001}$ ( $\sim 0.5$ KB) | $2^{4001}$ ( $\sim 0.5$ KB)           |
| 1000 root siblings ( $\delta=t$ )    | — (ties)                   | $\max 2^{1003}, 125$ KB      | $\max 2^{23}, 5$ KB         | <b><math>\max 2^{20}, 4</math> KB</b> |

**The carrier.** The natural representation is a **parent-relative bit string per node**: the fold is concatenation, comparison is lexicographic ( $O(1)$ -ish with structure sharing); the rationals are the semantics of the strings, kept for the proofs. Nothing here computes  $M(\delta)$ : the endpoints  $0.b + w$  and  $0.b + 2w$  are the strings  $b01$  and  $b10$ , so the mint emits a block and the fold concatenates, and  $hi$  need not be stored at all. The Lean mechanization drops even the framing bits: its coordinate is the bare codeword concatenation on `List Bool` (mint appends  $\Gamma.\text{enc}(t - a)$  to the anchor’s coordinate), with the prefix rule stated directly on strings; the quarter framing serves the interval realization alone, buying positivity and subtree interiority. The rational  $M(\delta)$  is computed in exactly one place, the validation model `embed_tree.py`, as exact fractions; the entropy tables read their bit counts off those denominators. IEEE 754 binary64 is machine-refuted as the carrier, a faithfulness failure rather than a design failure: mints degenerate at  $t = 52$ , depths underflow at 44, and rounding voids Theorem 15.1(iii) (6/120 PBT executions die). The obstruction is a pigeonhole (chains carry  $\Theta(\sum \log \delta)$  bits; a fixed-width word holds 64), but binary64 is sound as a *comparison cache* with exact fallback on float equality.

**Timestamps in practice.** The unique stamps in  $\mathbb{N}^+$  are Lamport’s 1978 construction, flattened: each replica ticks a counter per local operation, advances it to the max on merge, and breaks ties by replica id, packed as  $t \cdot 2^k + r$  with  $k$  id bits. Uniqueness and causality ( $t_a < t_x$  gives  $\text{packed}_a < \text{packed}_x$  regardless of ids, so  $\delta \geq 1$ ) both survive the packing; the proof layer only *assumes* uniqueness (the Lamport-structural part of honesty), and this construction is the witness that the assumption is realizable. Two entropy caveats. (i) Packing inflates every delta by  $2^k$ , costing  $\sim 2k$  extra bits per chain level, which turns the 0.5 KB sequential run above into  $\sim 3$  KB at  $k = 10$ . Cheaper: extend the mint’s code to the pair,  $C(\Delta t)$  followed by a  $k$ -bit id. The pair code is still order-preserving prefix-free (lex on  $(\Delta t, r)$ ), and costs  $k$  bits only where the level is minted. A further option, unvalidated against the battery and recorded as a design note, is to order sibling ids *relative to the anchor’s* (any deterministic per-anchor id order is a legitimate tiebreak; convergence needs agreement, not a uniform order), making “same author as my anchor”, i.e. sequential typing, cost  $\sim 1$  bit, so only genuine cross-author races pay  $\log R$ .

(ii) The clock must be *dense logical time, never physical*: wall-clock or HLC stamps make  $\delta$  measure elapsed time rather than elapsed events: a keystroke every 100 ms over  $\mu$ s-resolution stamps mints  $\delta \approx 10^5$ ,  $\sim 34$  bits per level for a purely sequential edit. The entropy law holds precisely because the counter ticks once per event, making  $\delta$  an event-count gap; if wall-clock order is ever wanted, it belongs in a display-layer tiebreak, never in the minted coordinate.

**What remains open.** Dead chain entries concatenated into survivors' strings are the conservation law's irreducible content (§14); reclaiming them under *causal stability* (renumber once every replica has seen the delete, the only condition under which rewriting geometry is safe, since no verdict involving the dead can be contested again) is the deepest of the open cost items, and it is orthogonal to correctness. The wider compression headroom is catalogued with its research questions in the companion assessment `whiteboard/order-coding-compression-notes.md`: context-conditioned per-anchor codes (licensed by comparison locality: a chain-lex first difference never crosses anchors), coalescing  $\delta=1$  runs, Steiner-sharing dead prefixes across co-heirs, stability-gated rank re-coding, and the Elias- $\delta$  constant (step 2 above).

## 16 The insert prefix: for the proof, and the proof alone

The insert payload  $\pi$  exists for verification, not execution. This section says why it must exist, and why an implementation may drop it from the wire.

**Why rc must be Either.** The verification framework (Part I) resolves a concurrent pair of non-commuting operations by consulting a declared conflict-resolution order  $rc$ ; any *oriented* entry for an insert/delete pair, however, incurs a base commutation obligation that is machine-refuted for this design space: ordering  $\text{ins}(x \leftarrow a)$  against  $\text{del}(a)$  (in either direction) is undischageable as a conflict entry, because rehoming is non-transitive across chains of deletions. So  $rc = \text{Either}$ : the proof must show the two *commute*, in both orders, on all applicable states.

**The commutation, and where the prefix earns its keep.** Order 1,  $\text{ins}(x \leftarrow a)$  then  $\text{del}(a)$ :  $x$  is born with chain  $\pi \cdot x$ , and the delete touches no chain (downstairs: the fold re-homes  $x$ 's record with composed coordinates; by exactness  $\alpha(x) = \alpha(a) \circ M(x - a)$ ). Order 2,  $\text{del}(a)$  then  $\text{ins}(x \leftarrow a)$ :  $a$  is gone; application must still be *total* and land  $x$  on the same sort key. This is what the carried prefix  $\pi = \text{chain}(a)$  (equivalently, the anchor's coordinate) provides: apply sets  $\text{chain}(x) = \pi \cdot x$  (downstairs:  $\alpha(x) = \pi \circ M(x - a)$ , attached to the nearest live ancestor). The two orders produce *equal* states: commutation to equality, not merely up to  $\approx$ , courtesy of canonicity (Theorem 14.1). Figure 9 draws the square on the smallest instance.

**Ghost, in the strict sense.** On every reachable state the anchor is live at application (honesty), and the application reads only the state and the two timestamps:  $\pi$  is never consulted. The Lean artifact carries this as `ins_prefix_ghost` (`Sal/MRDTs/RGA_Embed/RGA_Embed_MRDT.lean`): on accurate ops the always- $\pi$  transition and the state-reading transition are literally the same function: the two conventions of this section coincide on every honest state. An implementation may therefore drop  $\pi$  from the wire entirely; the *proof* may not, because the verification conditions quantify over reorderings that visit unreachable states, and there application without  $\pi$  is partial. The prefix costs  $O(\sum \log \delta)$  bits, the same entropy as the coordinate itself, and exists at the proof layer alone.

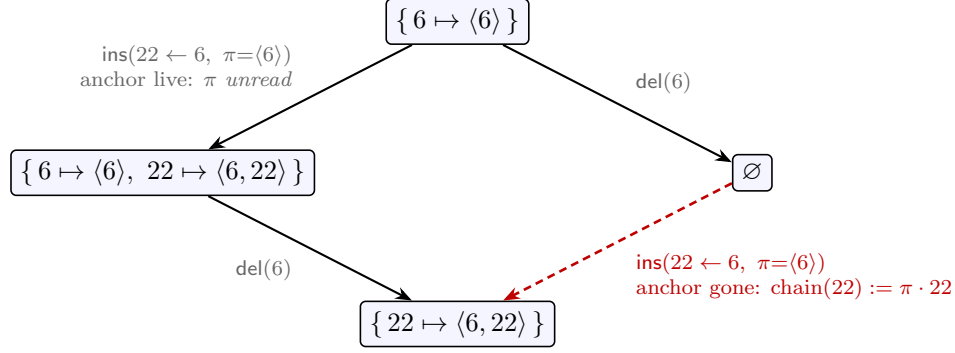


Figure 9:  $rc = \text{Either}$  discharged:  $\text{ins}(22 \leftarrow 6)$  and  $\text{del}(6)$  commute *to equality*. Down the left, the anchor is live and application never consults  $\pi$  (the reachable, honest order). Down the right, 6 is already gone: the state cannot supply the anchor’s chain, so the carried  $\pi = \text{chain}(6)$  does, landing 22 on the same sort key. The right-hand state is unreachable under honesty; the verification condition quantifies over it anyway, and  $\pi$  exists exactly to make that corner total.

## 17 Validation

All numbers reproduce from `whiteboard/litmus/` in the accompanying Sal repository (`embed_tree.py`; harnesses `litmus.py`, `pbt.py`). Notation:  $\text{RGA}_\dagger$  is the published tombstoned RGA (Roh et al.; §20); the *rehoming* RGA is the convergence-verified flat tombstone-free design of §14 (since demoted repo-wide: the L1 row below was generalized into the kernel-checked sequential refutation of Part III, §27, and this design took the canonical seat). The L1–L25 battery is the litmus suite accumulated over this project’s design sweep; each entry is the named countermodel that killed at least one of 16 prior candidate designs. The one exception below, L19, is discharged by the sided generalization of §18.

| check                                                                                                                                                                          | verdict                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| litmus battery L1–L25 (incl. every countermodel that killed 16 prior designs: ceiling escapes, tie inheritance, three-branch topology, frame mixing, fold-verdict inheritance) | clean, except one-sided L19 (backward-run interleaving, the published RGA’s own anomaly) |
| credential countermodel (tie heirs vs mid-ts third party, death-before vs death-after topologies)                                                                              | converges, no flips;<br>= $\text{RGA}_\dagger$ ’s reads                                  |
| subordination-escape and retroactive-subordination CEs                                                                                                                         | survive                                                                                  |
| randomized version-DAG PBT (FLIP / CONV / LIVE / DUP)                                                                                                                          | 120/120 and 300/300 clean                                                                |
| per-read sibling-disjointness assertion sweep                                                                                                                                  | 320/320 clean                                                                            |
| chain-lex instrumentation (19,531 sibling groups)                                                                                                                              | 0 violations                                                                             |
| <b>lockstep vs the published tombstoned RGA</b>                                                                                                                                | <b>read-equal at every</b>                                                               |
|                                                                                                                                                                                | <b>apply/merge/read, 120/120</b>                                                         |
| L1 delete-reorder vs the verified rehoming RGA                                                                                                                                 | embed $[2, 3]$ ✓ vs<br>rehoming $[3, 2]$ ×                                               |

The lockstep row is the identification: *the embedded-chain RGA is observationally the published tombstoned RGA with the tombstone set compressed into coordinate entropy*, machine-checked on 120 executions and now a kernel-checked theorem (the compaction theorem, §21): on every honest event set, any enumeration of the tombstoned RGA’s visible ids in its own visible order *equals* the embed read, element for element. The L1 row is the separation from the verified

rehoming RGA, and it is exactly the one-line difference of §14: that design’s climb re-sorts a rehomed node by its own timestamp (the  $[b, a, c] \rightarrow [c, b]$  anomaly), i.e. its sort key is the birth chain *truncated at each rehoming*; this design keeps the chain whole.

## 18 The sided generalization: two-sidedness as a parameter

The one blemish in the battery is L19: two replicas that each type a *backward* run at the same place (repeatedly “insert before the current head”) interleave, because every insert-before anchors at the same left neighbor and both runs land in one sibling group. The fan is not a traversal artifact: no read order of the same birth tree un-fans a fan, so any fix must change where insert-before *anchors*. The Fugue family’s answer is a second side: insert-before anchors at the *right* neighbor as an L-entry, so a backward run is a chain, exactly as a forward run already is.

**The L19 example, concretely.** It is the battery’s own trace (Figure 10). The LCA document is  $\langle 1 \rangle$  (one insert, stamp 1, anchored at the start marker  $\diamond$ ). Then, concurrently, replica A types a backward run at the head, three inserts each anchored “before the current first element”:

$$\text{ins}(10 \text{ at head}), \quad \text{ins}(30 \text{ at head}), \quad \text{ins}(50 \text{ at head}),$$

so A locally reads  $\langle 50, 30, 10, 1 \rangle$ ; replica B does the same with stamps 21, 41, 61 and locally reads  $\langle 61, 41, 21, 1 \rangle$ . (The stamps interleave numerically,  $10 < 21 < 30 < 41 < 50 < 61$ , because the two replicas’ Lamport clocks advance together.) In the one-sided datatype, “insert at the head” can only anchor at the left neighbor, which is  $\diamond$  for every one of the six inserts: all six become R-children of  $\diamond$ , one sibling group, and the sibling tiebreak (newest first) merges the two runs by stamp:

$$\langle 61, 50, 41, 30, 21, 10, 1 \rangle$$

after the merge, the two sentences shuffled together. No traversal of that tree can do better; the information “these three form one run” is simply not in a sibling fan whose stamps interleave.

**The sided datatype, and why it prevents the interleaving.** The generalization does not build a second datatype. An insert names an anchor *and a side*; chains become sequences of (side, timestamp delta) entries; and the display rule generalizes from the prefix rule (“a prefix precedes its extensions”) to the in-order rule (“L-extensions, then the node, then R-extensions”). One marker trick makes this ordinary lexicographic comparison again: a node’s sort key is its coordinate followed by a virtual terminator, and the sided symbol alphabet is ordered

$$(R, 1) < (R, 2) < \dots < \text{marker} < \dots < (L, 2) < (L, 1),$$

with the L band *mirrored* (among L-siblings the newer sits adjacent to the node). Marker above the R band puts a node before its R-extensions; marker below the L band puts L-extensions before the node.

On the example, the Fugue anchoring rule (the generation-policy paragraph below) sends each “insert at head” to the *successor* as an L-entry, so each run is an L-chain (Figure 10, right). Writing chains as entry sequences with deltas relative to the parent’s stamp ( $10 = 1 + 9$ ,

30 = 10 + 20, ...):

$$\begin{array}{ll} \text{chain}(1) = (R, 1) & \text{chain}(10) = (R, 1) (L, 9) \\ \text{chain}(30) = (R, 1) (L, 9) (L, 20) & \text{chain}(21) = (R, 1) (L, 20) \\ \text{chain}(50) = (R, 1) (L, 9) (L, 20) (L, 20) & \text{chain}(41) = (R, 1) (L, 20) (L, 20) \end{array}$$

and the marker-lex comparisons (keys descending in display order) work out as follows, writing  $M$  for the marker:

- *L-extensions precede the node*:  $\text{key}(10) = (R, 1)(L, 9) M$  vs  $\text{key}(1) = (R, 1) M$  first differ at position 2,  $(L, 9)$  vs  $M$ , and the L band sits above the marker: so 10 displays before 1. Likewise 30 before 10, 50 before 30: each backward keystroke lands immediately left of the previous one, which is the typed intent.
- *The two runs are decided once, not per pair*:  $\text{key}(10)$  vs  $\text{key}(21)$  first differ at position 2,  $(L, 9)$  vs  $(L, 20)$ , and the L band is mirrored, so  $(L, 20) < (L, 9)$ : A's run displays before B's. Now *every* node of A's chain carries the prefix  $(R, 1)(L, 9)$  and every node of B's carries  $(R, 1)(L, 20)$ , so *every* cross-pair first-differs at that same position 2 with that same verdict. The blocks cannot interleave:

$$\langle 50, 30, 10, 61, 41, 21, 1 \rangle.$$

This is the datatype-level theorem **schain\_subtree\_convex** (a subtree is an in-order interval; §21): *any* policy that keeps a run inside one subtree gets non-interleaving from the datatype, for free.

The current datatype is the all-R fragment of this order, *exactly*: with L uninhabited the in-order rule degenerates to the prefix rule, and the existing key of §15 (the sym-mapped bits plus terminator) is literally this construction with the L band empty. That is a theorem, not a slogan: the mechanization's fragment theorem (§21) proves all-R chains order exactly as one-sided chains.

**How the encoding realizes it, for zero extra bits.** The one-sided block of §15 spends a constant leading 1 per level whose only job is keeping coordinates nonempty; repurpose it as the side bit. The L band needs an order-reversing prefix-free code, and the bit complement  $\overline{C}$  is exactly that at identical lengths (flipping every bit flips every first-difference verdict and preserves both lengths and prefix relations), so the block becomes

$$\text{block}(\text{side}, \delta) = \text{sideBit} \parallel (C(\delta) \text{ if } R, \overline{C}(\delta) \text{ if } L),$$

prefix-free per band, with no framing change and no new bits. On the example, with the binary delta code ( $C(1) = 0$ ,  $C(9) = 1110001$ ,  $C(20) = 111100100$ , so  $\overline{C}(9) = 0001110$ ,  $\overline{C}(20) = 000011011$ ), the stored coordinates are the concatenated blocks:

| node | chain                        | stored coordinate (side bits marked)      |
|------|------------------------------|-------------------------------------------|
| 1    | $(R, 1)$                     | <u>1</u> 0                                |
| 10   | $(R, 1)(L, 9)$               | <u>1</u> 0 00001110                       |
| 21   | $(R, 1)(L, 20)$              | <u>1</u> 0 0000011011                     |
| 30   | $(R, 1)(L, 9)(L, 20)$        | <u>1</u> 0 00001110 0000011011            |
| 50   | $(R, 1)(L, 9)(L, 20)(L, 20)$ | <u>1</u> 0 00001110 0000011011 0000011011 |



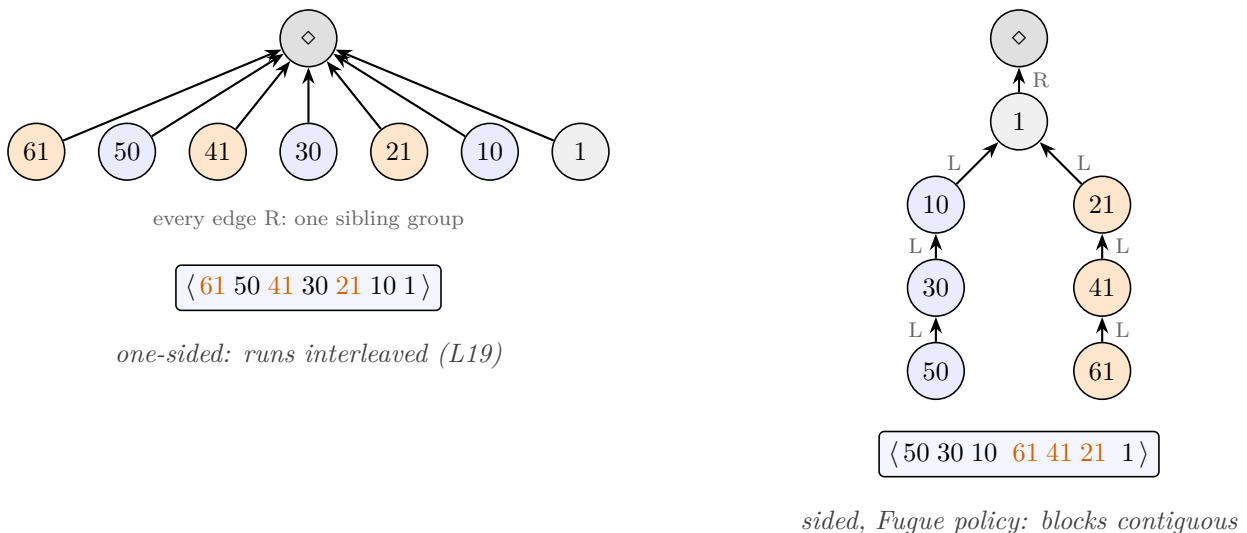


Figure 10: L19 under the two anchoring rules, the battery’s own trace. LCA document  $\langle 1 \rangle$ ; replica A (blue) types backward 10, 30, 50 before the head while replica B (orange) concurrently types backward 21, 41, 61. One-sided (left): every insert-before anchors at the same left neighbor  $\diamond$ , both runs land in one R-sibling group, and the timestamp-sorted fan interleaves them. Sided under the Fugue policy (right): insert-before anchors at the *successor* as an L-entry, each run is an L-chain hanging off 1, and the in-order read keeps the blocks contiguous: L-extensions precede the node (50, 30, 10 before their parents’ line ending at 1), and among the L-siblings 10, 21 of node 1 the older run displays first (the L band is mirrored). Both display strips are machine outputs of `embed_sided.py`.

To compare two coordinates, parse: read a side bit; if 1, decode one  $C$  codeword as an R-band symbol  $(R, \delta)$ ; if 0, decode one  $\bar{C}$  codeword (decode  $C$  on the flipped bits) as an L-band symbol  $(L, \delta)$ ; repeat, then terminate with the marker. Prefix-freedom per band delimits every block, so no length fields or framing are needed, exactly as one-sided. The complement is what makes the mirror free: within the L band, plain descending comparison of the *stored* bits is already the mirrored delta order ( $\bar{C}(20) = 000011011 < \bar{C}(9) = 0001110$  bitwise, matching  $(L, 20) < (L, 9)$ ), so the carrier stores entropy payload only and the marker-lex of the previous paragraph is read off the bits. Faithfulness of this parse (unique decodability plus order embedding of the sided alphabet) is machine-checked by the chain-model/code-model lockstep below.

Two honest cost notes. Runs absorb the side bit like every other run-constant (paid once at the head). On *this* trace the sided coordinates are longer than the one-sided fan’s (the runs’ deltas are 20 because the two replicas’ Lamport clocks interleave); the compression claim is about the common case, continuation-stamped backward typing ( $\delta = 1$  per keystroke, an  $0\|\bar{C}(1) = 01$  block per level), where today’s design pays a growing- $\delta$  fan. Two-sidedness buys the semantics always and the compression exactly when backward runs are locally stamped.

**Side selection is a generation policy, not a datatype choice.** If insert merely “takes a side,” convergence and RA-linearizability hold for any sides whatsoever: those proofs never consult the order’s intent. What depends on *which* side is picked is the ordering intent. Fugue’s non-interleaving needs the rule “right child of the left neighbor when it has no right children, else

left child of the successor,” and that rule lives at generation time, where the framework already has a slot: honesty, the same layer that polices the mint. The architecture is therefore one sided datatype, verified once, policy-free (canonicity, fold exactness, the Join, RA-linearizability), with intent theorems per generation policy: under always-R, read-equivalence to the published RGA (the fragment theorem composed with the compaction theorem); under the Fugue rule, non-interleaving (a subtree is an in-order interval). RGA and Fugue become two disciplines over one kernel rather than two datatypes.

**Validation.** `whiteboard/litmus/embed_sided.py`: the chain semantics and the bit carrier as lockstep twin models, both run under both policies. Every design prediction survived the first full run:

| check                                                                                            | verdict                                                                             |
|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| full gauntlet under the Fugue policy (L1–L25, credential family, subordination CEs, stale forks) | clean, <b>including L19</b><br>(out = $\langle 50, 30, 10, 61, 41, 21, 1 \rangle$ ) |
| same gauntlet under always-R                                                                     | clean except L19: the<br>one-sided fan, reproduced                                  |
| chain semantics $\sim$ bit carrier, both policies                                                | lockstep-EXACT (unique<br>decodability + order embedding)                           |
| all-R fragment $\sim$ the one-sided embed model                                                  | lockstep-EXACT, every scenario                                                      |
| randomized version-DAG PBT, all four models                                                      | 120 + 300 clean                                                                     |

The Lean mechanization of both layers (the sided chain-lex kernel and the sided conditioned instance, capstone included) has landed kernel-clean; §21 records it.

## 19 Comparison with Eg-walker

Eg-walker (Gentle & Kleppmann, EuroSys 2025) is the closest system in spirit, and the two designs can be read as the *operational* and *denotational* realizations of one thesis, **pay for concurrency, not for history**:

|                         | Eg-walker                                                                                                                                     | embedded-chain RGA                                                                                                                                                                                |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| model                   | op-based; replicas exchange events                                                                                                            | MRDT: three-way merge with an LCA                                                                                                                                                                 |
| durable state           | <b>full event graph on disk</b> , deletes included, retained while concurrent ops may arrive; document state metadata-free                    | <b>live nodes only</b> ; no event log anywhere; dead survive as $\Theta(\log \delta)$ bits inside live coordinates                                                                                |
| merge mechanism         | <i>replay</i> : walk the event graph, rebuild a transient CRDT structure (tombstones reappear transiently), discard at causally-linear points | <i>refold</i> : $O(\text{survivors})$ coordinate recomputation; no replay, no transient structure, no order decisions                                                                             |
| sequential-editing cost | $\approx$ zero (plain edits; no metadata)                                                                                                     | $\approx$ zero order metadata ( $\sim 4$ bits/level for continuation typing; a cursor jump pays log-gap once, at the run head)                                                                    |
| concurrency cost        | replay time proportional to the concurrent region, at every affected merge                                                                    | coordinate bits proportional to $\log(\text{ts gap})$ , once, in space                                                                                                                            |
| ordering semantics      | Yjs-variant (FugueMax-adjacent); maximal non-interleaving <i>conjectured</i> , not proved; two-sided (no L19 analogue)                        | $\equiv$ published RGA (machine lockstep); one-sided: L19 backward-run interleaving inherited; removed by the sided generalization (§18), RGA kept as the all-R fragment                          |
| correctness artifact    | informal proof (Appendix C) + property testing                                                                                                | L1–L25 battery + DAG PBT + lockstep vs RGA <sub>†</sub> ; <b>RA-linearizability kernel-checked in Lean</b> (§21); the design was chosen so the merge proves itself (no order decisions to verify) |
| op payload              | integer origin positions (compact, human-meaningful)                                                                                          | two timestamps + ghost prefix (droppable on the wire; §16)                                                                                                                                        |

Four sentences of honest positioning. *(i)* Eg-walker achieves its clean document state by keeping the entire history and re-deriving order on demand; the embedded-chain RGA achieves it by making order derivation unnecessary: the sort keys are written at birth and never edited, and the coordinates are their encoding (§15), precomputed at mint time and exact forever. *(ii)* Eg-walker’s ordering target was stronger where L19 is concerned (Fugue-style two-sidedness removes backward interleaving, while the one-sided design inherits RGA’s anomaly by construction, being read-equal to RGA); the sided generalization of §18 adopts exactly that two-sidedness as a parameter, closing the gap while keeping the one-sided datatype as its all-R fragment. *(iii)* The verification gap runs the other way: Eg-walker’s replay-transform-clear loop is algorithmically sophisticated and only informally proved; here the merge performs no arbitration at all, the metatheory is one-line at the chain layer plus a single faithfulness theorem at the encoding layer, and RA-linearizability is a kernel-checked theorem in a framework that had already verified a dozen-plus replicated datatypes end-to-end (§21). *(iv)* The cost models part company at cursor jumps: a single-user history that hops back to type at an old anchor is free for Eg-walker (nothing concurrent to replay) but mints log-gap bits here, once per jump, at the run head; on measured

single-user editing traces the jump tail is thin (code bits  $\approx 1.4$ – $1.8$  per level; see the companion note of §15). In the MRDT setting there is also a model fit: Eg-walker must reconstruct merge context by replay because the op-based model gives it none; the MRDT model hands the merge its LCA, which is exactly the context replay rebuilds.

## 20 Related work

The verified sweep of nearby systems lives in `whiteboard/litmus/related-work.md` (per-claim verdicts, primary sources); this section positions the datatype.

**Retention taxonomy.** What nearby systems keep for the dead: tombstone records (RGA with cemetery under causal stability; WOOT; Yjs’s per-deletion GC structs, kept forever; Fugue, “we cannot remove a deleted element’s node entirely”; Peritext, “the positions must be remembered”); the full history (Eg-walker’s event graph, Chronofold/causal trees where the log *is* the text); consensus-gated compaction (Treedoc’s flatten requires Paxos-commit); or **dead identifiers inside live position keys** (Logoot/LSEQ, Weidner’s position-strings/list-positions, the Fugue id space). The embedded-chain RGA is in the last class *by information content* (dead ancestors’ timestamps persist inside survivors’ sort keys, at live-reachable scope), but with three deltas to its classmates: the retention is *delta-coded at the entropy of the anchor gaps* (a sequential history costs a constant per element where Logoot-family keys grow with allocation policy), the *birth tree kept whole* gives forward non-interleaving and delete-order preservation (position-key designs interleave, the PaPoC’19 anomaly, and Figma-style fractional indexing is unsound under concurrency by its own account), and the semantics is pinned to the best-studied sequence CRDT by a machine-checked lockstep rather than being a new point in anomaly space.

**The impossibility backdrop.** Attiya et al. (PODC 2016) prove  $\Omega(D)$  metadata for push-based protocols meeting even the weak list spec: the published form of “ordering requires remembering the dead.” This design does not escape the bound; it *relocates and minimizes* the memory:  $\Theta(\log \delta)$  bits per dead-chain element inside live coordinates (plus the MRDT version store, which the model provides). The sharper, machine-witnessed form from our design sweep is the *conservation law* (§14): the sort key must retain dead ancestors’ timestamps (the verdict information survives every encoding change: stamp, spine, tombstone, or denominator bits), and ties force separate *metadata* only under timestamp-oblivious minting; the timestamp-faithful mint carries the same bits inside the coordinate. Eight timestamp-oblivious mutation variants each have a named machine-checked countermodel (fold-verdict erasure; repair non-locality); the credential countermodel separates this design from all of them.

**Verified datatypes.** Isabelle RGA (Gomes et al. OOPSLA’17, op-based convergence), OpSets (list semantics specified by a total order of identified ops: arbitration-by-identity as specification), VeriFx (SMT, convergence properties), MET (TLA+, list CRDT bugs). In the MRDT line (Kaki et al. OOPSLA’19; Peepul PLDI’22; RA-lin OOPSLA’25, this framework), no verified sequence with sequence-CRDT-grade ordering guarantees existed; our verified rehoming RGA (§14) is RA-linearizable against its own fold yet fails the sequential delete-order spec, since RA-linearizability certifies convergence to the datatype’s *own* semantics and ordering intent must be specified and proved separately. The embedded-chain RGA delivers the missing artifact: RA-linearizability is a kernel-checked Lean theorem (§21); delete-order preservation, display stability and forward

non-interleaving are proved at the model layer; and the RGA read-equivalence (the compaction theorem, §21) closes the intent question against the best-studied sequence semantics.

**Treedoc’s lesson, inverted.** Treedoc re-balances geometry and must pay consensus for it; the principle that emerged from our ledger-based sibling design study (*reprojection without consensus is safe iff geometry has been demoted to a rendering*) is here taken to its fixed point: geometry is never reprojected at all. It is immutable, so it may safely be the only thing.

## 21 Mechanization: the conditioned discharge and the compaction theorem

The verification framework is Part I’s metatheory; file paths below are relative to the Sal repository.

**The capstone is proved** (2026-07-15, kernel-clean on {proptext, Classical.choice, Quot.sound}):

$$\text{embed\_ra\_linearizable3} : \text{EReach } \Gamma \ C \rightarrow \text{IsRALinearizable3 } C$$

i.e. RA-linearizability per version at every honestly reachable configuration, *parametric in the code*  $\Gamma$ : the unary code, the binary delta code, and the flipped Elias- $\delta$  code are all proved **OrderedPrefixCode** instances, three encodings on one proof. The factoring is exactly this part’s design–encoding split, which is itself evidence for the split:

1. **Model layer** (Sal/MRDTs/RGA\_Embed/, six files, 0 sorry, plus the read-equivalence order core and the sided kernel noted below): the code-parametric kernel; commutations *to state equality* under `rc = Either (rc_non_comm')`; `ins_prefix_ghost`; coherence and chain-generation closures; display stability, with step stability (S2, delete-order preservation at its `del` instance) proved *without any axioms* and merge pairwise stability (S4, the adopted contract); unique decodability (`coordOf_inj`); **the chain-lex theorem** (`display_iff_chainBefore` = Corollary 15.2); non-interleaving as subtree convexity (`subtree_convex`); Theorem 15.1(i) for all three codes (`binEnc_mono/binEnc_prefixFree`, `dEnc_mono/dEnc_prefixFree`, cross-validated against the Python codes by kernel `decide`); `native_decide` SPOTs where the rehoming RGA’s delete-reorder witness gets the opposite verdict.
2. **Conditioned instance** (Sal/ConditionedMRDTs/MRDT\_Instances/EmbedRGA/, nine sections, 0 sorry, plus the Elias- $\delta$  inheritance corollary `EmbedRGA_EliasDelta.lean`), proved via the *mergeable-queue route* of §13: a generic join lemma (`JoinLemma3At`: the merge exhibits its own linearization witness) plus an honest-reachability metatheorem (**HonestReach**), rather than the rehoming RGA’s 55-file bespoke chain. The route applies exactly when states are canonical per event set, which is Theorem 14.1(i). Landed: the canonical sorted-list state; strictly-sorted extensionality (`esorted_ext`: sorted lists with the same members are equal, replacing any canonical-list formula); **fold-canonicity** (`e_fold_canon`: well-formed enumerations of one event set fold to the *same* state; Theorem 14.1(i) mechanized); the honesty core and its bridge to well-formedness; the merge membership characterization (`e_mergeL_mem`, OR-set survival with honesty closing the

cross-branch-delete corner); the Join (`e_join_at`: the merge is its own linearization witness); the capstone; the honesty discharge from the framework’s generation discipline (`eHonest_of_applicable`: applicable generation forces the honesty core, so per-op honesty needs no bespoke oracle); and the instance intent theorems: delete-order preservation *is* sublist stability (`eUpdate_del_sublist` is literally `List.filter_sublist`: a delete’s successor state is a sublist of its predecessor, so the surviving display order is untouched; the merge preserves each input’s survivor order on both sides).

**The compaction theorem is proved** (2026-07-15, kernel-clean):  $\text{read} \circ \text{embed} = \text{read} \circ \text{RGA}_\dagger$  on honest executions, as `rga_read_eq_embed_read` (`EmbedRGA_ReadEquiv.lean`): for every well-formed enumeration of an honest event set, *any* enumeration of the tombstoned fold’s visible ids in the published RGA’s own relational order (`visible_lt`) equals the embed fold’s id sequence, with element agreement (`embed_rec_readSeq`). The proof is *against the published definition*: the order core (`Sal/MRDTs/RGA_Embed/RGA_Embed_ReadEquiv.lean`) shows `visible_lt` coincides with chain-lex on the shared birth tree (soundness by induction on derivations, completeness by realizing every chain prefix as an `after`s-reachable ancestor), and en route proves the published relational order total and asymmetric on the birth tree, facts the published side never had. This converts the design into a theorem *about* the published RGA: it admits a strictly-live compaction preserving every read. The lockstep rows of §17 are this theorem’s tested form.

**The sequential specification is proved, and the design holds the canonical seat** (2026-07-16). The campaign of Part III proved `embed_seq_sound`: on every sequentially honest single-replica history the canonical sorted state is the naive insert-at-position text buffer, record for record, with the adjacency lemma (`chainBefore_snoc_iff`: a fresh insert’s delta beats every existing child’s by sum-telescoping) as the geometric heart (§26). The published  $\text{RGA}_\dagger$  inherits the same buffer spec through the compaction theorem (`rga_seq_read_eq_buffer`), and the same statement is refuted for the rehoming design (§27), which demoted it repo-wide: the embedded-chain family is the repository’s canonical sequence RDT, and the canonical fused Peritext was re-based onto this kernel (`Peritext_Embed`/: capstone by pure instantiation of the payload-generic instance; `renderIds_del`, deleting a character never re-formats another, the theorem the rehoming kernel provably lacks; and the tier-4 editor theorem `peritextEmbed_seq_sound`). One caveat is recorded as Open Question 31.9: per-replica soundness and convergence do not compose into “the merged read is the naive buffer on some ordering of the union”; the dead-anchor corner (§29) is where stamp-sorted replay diverges from the fold.

**The sided generalization is proved** (2026-07-15, kernel-clean), in the same two layers as §18 presents it. The sided chain-lex kernel (`Sal/MRDTs/RGA_Embed/Sided_ChainLex.lean`): the sided marker theorem `sdisplay_iff_schainBefore` (display comparison of sided coordinates IS the in-order rule), totality of the display relation *without any axioms*, unique decodability (`sidedCoordOf_inj`: the side is recoverable from the first symbol’s band, the delta by prefix-freedom within the band), the all-R *fragment theorem* `schainBefore_liftR` (all-R chains order exactly as one-sided chains: “RGA is the all-R fragment,” mechanized), and in-order-interval subtree convexity (`schain_subtree_convex`, the datatype half of the L19 discharge). The conditioned instance (`MRDT_Instances/SidedRGA`/, nine sections mirroring the one-sided instance with the sided key): fold-canonicity (`s_fold_canon`), the Join by the same mergeable-queue route, and the capstone

$$\text{sided\_embed\_ra\_linearizable3} : \text{SReach } \Gamma \ C \rightarrow \text{IsRALinearizable3 } C$$



for every side assignment: sides are payload to convergence, which is the policy/datatype split of §18 as a theorem, with `sHonest_of_applicable` discharging honesty from the generation guard exactly as one-sided. The per-policy intent theorems have since landed, and the policy story is a machine-checked three-act arc (2026-07-16/17). *Act one:* `sFold_liftOp`, the all-R sided fold IS the one-sided fold, unconditionally (the sort keys coincide definitionally), so every one-sided theorem transports; and `sided_fold_subtree_convex`, subtrees display as contiguous blocks in any chain-generated fold, so runs that chain never interleave (`fugue_reachable_runs_no_interleave`). *Act two:* the Weidner–Kleppmann maximal-non-interleaving Definition 4 was formalized over reachable configurations (`SidedRGA_Fugue.lean`) and REFUTED twice for this Fugue policy on reachable traces (the recency tiebreak reverses the paper’s lowest-ID-first; their Figure-7 execution): the paper’s own Fugue-vs-FugueMax separation, landing on this implementation. *Act three:* FugueMax realized as a kernel variant (`SidedMax_ChainLex.lean`): its sibling order depends on the right ORIGIN, not expressible by any re-banding of deltas (the mirror control machine-witnesses this), so R-entries carry the mint-time successor’s key as an order-reversing tag; condition (3) is proved in full (`fuguemax_same_origin_low_first`, the positive twin of the refutation) and full Theorem 9 is reduced to the display-to-traversal theory (task #88; Open Question 31.10). The sided sequential spec also landed (`sided_seq_sound`: a sentinel-rooted two-sided buffer, R-inserts splice after their anchor, L-inserts immediately before). The tag has a COST: at contested inserts the coordinate embeds the successor’s key,  $\Theta(|\text{key}|)$  where plain Fugue pays zero, and whether that retention is a lower bound for tombstone-free maximal non-interleaving is an open question with a Myhill–Nerode shape (task #89; Open Question 31.11): the entropy law of §15 meeting the ordering intent.

**Provenance.** `whiteboard/litmus/embed_tree.py` (the design, the timestamp-oblivious control, the credential countermodel, the IEEE 754 measurements; `python3 embed_tree.py / fp / code`); `litmus.py`, `pbt.py` (battery and DAG harness); `contest_tree.py` (lockstep harness, CE regressions); `whiteboard/retention-countermodel.pdf` (the retention arc that forced this design); `whiteboard/delta-tree-design.pdf` (the ledger-based sibling design and the invariants I1–I3); `whiteboard/litmus/related-work.md` (verified source list: Roh et al. JPDC’11; Attiya et al. PODC’16; Kleppmann et al. PaPoC’19; Weidner & Kleppmann TPDS’25 (Fugue); Gentle & Kleppmann EuroSys’25 (Eg-walker, arXiv:2409.14252); Preguiça et al. ICDCS’09 (Treedoc); Grishchenko & Patrakeev PaPoC’20 (Chronofold); Litt et al. CSCW’22 (Peritext); Gomes et al. OOPSLA’17; OpSets 2018; Kaki et al. OOPSLA’19; Peepul PLDI’22; RA-lin OOPSLA’25; VeriFx ECOOP’23; Figma/Wallace fractional indexing; Weidner position-strings).

## Part III

# Sequential specifications: convergence is not correctness

Replication-aware linearizability certifies that every version of a replicated data type is the fold of some delivery-respecting linearization of its events. The reference point of that guarantee is the datatype’s *own* fold: a bug in `do` appears identically on every replica, converges perfectly, and passes every convergence theorem. This part records the campaign that closes the gap. For



every production RDT in Sal there is now a kernel-checked theorem that the fold of any single-replica history equals the output of a straightforward sequential program, the one a programmer would write with no replication in mind. The campaign’s unifying observation is the *witness discipline*: sequential interfaces address state positionally (`dequeue()` is nullary, `insert` names an index), replicated operations cannot, and every replicated operation therefore carries the identity of what its issuer *observed* the positional referent to be; the honesty side conditions of the theorems are exactly the faithfulness of those witnesses. The campaign’s negative results carry as much information as its theorems: the rehoming tombstone-free RGA is refuted at this specification on a four-op single-replica trace, its fused Peritext inherits the refutation at the render, and Shesha passes sequentially while failing at the merge, so do-faithfulness and merge-faithfulness are independent axes, and both separations are kernel-checked. A final section states what the campaign does *not* give: per-replica soundness and convergence do not compose into “the merged state is some user’s sequential result,” and the precise corner where composition fails is the dead-anchor corner the sequence-CRDT trilemma names. Every claim below names its Lean identifier.

## 22 The specification gap

The framework’s central guarantee is RA-linearizability at every reachable configuration: each version the store registers is  $\text{fold}(\rho)$  for some enumeration  $\rho$  of the version’s event set that respects delivery order. This is the right guarantee for *replication*: the merge invented nothing, reordered nothing it was not entitled to reorder, and all replicas agree.

It says nothing about whether the fold is *right*. The reference sequence is the datatype’s own `do`; if `do` is wrong, every replica is wrong in unison (Figure 11). Two incidents made this concrete in this repository. First, a mark-extent defect in an early Peritext render passed every convergence obligation and was caught only by eye. Second, and decisively: the rehoming tombstone-free RGA carries a fully general, kernel-clean RA-linearizability capstone (`rga_ra_linearizable3_eq`), and its `do` nonetheless reorders surviving text when an interior element is deleted, with no concurrency anywhere in the history (§27). Both defects are invisible to convergence because convergence is self-referential.

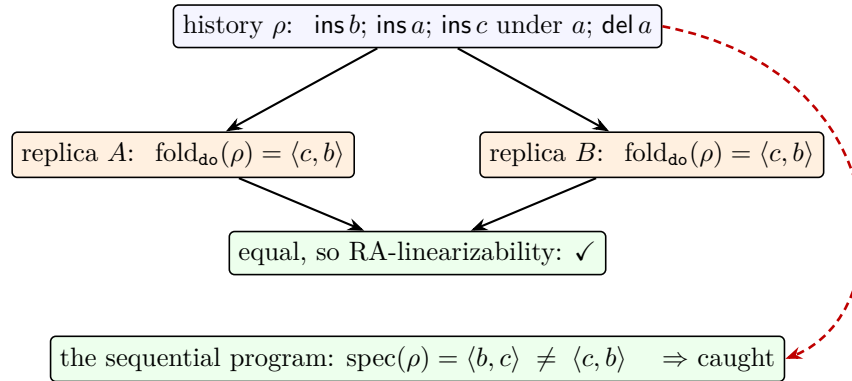


Figure 11: Why convergence cannot see a `do` bug. A defective fold is applied identically everywhere, so all replicas agree and every convergence theorem passes over the shared mistake. Only a reference computed by something *other than* `do` (dashed) can disagree. The history shown is the real refutation trace of §27.

The remedy is an independent specification. For sequential behaviour there is a canonical one: *the sequential program a programmer would write for the same operations with no replication in mind*. A counter is addition. A queue is a list. A text buffer is insert-at-position and delete. A marked-text editor is a buffer of characters and boundary tokens with one left-to-right walk.

## 23 The obligation, and the rules of engagement

The campaign’s obligation, per datatype:

$$\text{for every sequentially honest single-replica history } \rho: \\ \text{read}(\text{fold}(\rho)) = \text{spec}(\rho)$$

proved by discovering an inductive invariant of the fold, never by weakening the spec to match the implementation. “Sequentially honest” is the datatype’s own issuing discipline specialized to a linear history (fresh, monotone stamps; each operation applicable at the state it executes against), so the theorems attack no strawman: the histories quantified over are exactly the ones a single well-behaved replica produces, and in every case the side condition reuses a contract the convergence proofs already consume.

Three method rules were fixed in advance and held throughout. *Falsifiable statement first*: each tier states its equality before proof work begins, and a refutation is a first-class outcome (three landed, §27). *The spec is external*: the sequential program is written from the datatype’s user-facing intent, never read off the implementation; where the two disagree, the disagreement is the result. *Tests carry both polarities*: every concrete test block pairs its expected values (hand-derived, never evaluated out of the code under test) with should-fail companions, so the harness demonstrably discriminates; the refutation files pair each negative with the positive prefix on which the two sides still agree, isolating the departing operation exactly.

## 24 The observation gap: positional interfaces, witnessed ops

Writing out the specifications exposes one structural pattern, and it is the campaign’s most instructive artifact. A sequential structure addresses its state *positionally or implicitly*: `dequeue()` takes no argument because “the head” is unambiguous; `insert(i, e)` names an index; `write(v)` implicitly overwrites everything before it. A replicated datatype can afford none of this, because positions and implicit referents are not stable under concurrent edits: by the time an operation is applied elsewhere, “the head” may be a different element and index *i* a different character. So the replicated operation carries a **witness**: the identity (tag, stamp, id, coordinate, snapshot) of what the issuing replica *observed* the positional referent to be, at its own state, at issue time (Figure 12).

The sequential-honesty side conditions of §23 are then exactly the statement *the witness is what the sequential operation would have seen*, and each campaign theorem says: under that discipline, the witnessed-identity program implements the positional program. Where no witness is needed the alphabets coincide and the spec is a view homomorphism. The whole catalogue of §25 sorts by this principle, and the summary is one sentence: *every honesty condition in the campaign is the faithfulness of a witness that replaced a positional argument*.

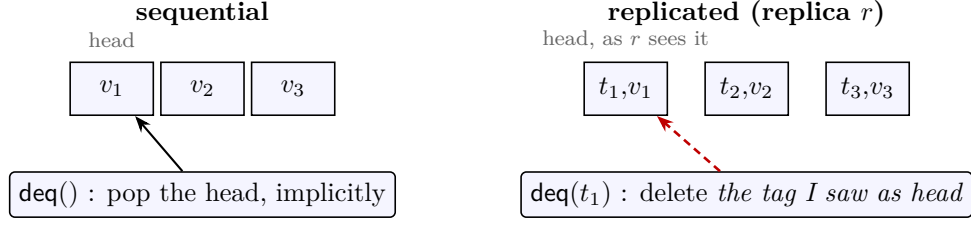


Figure 12: The observation gap, on the queue. The sequential **dequeue** is nullary; the replicated one names the tag its issuer read as the head of its own replica (dashed: the witness). Honesty is the faithfulness of the witness: **qOK** states that the fold at the issue point is literally  $(t_1, v_1) :: \text{rest}$ .

## 25 The specifications, datatype by datatype

### 25.1 No gap: the alphabets already coincide

For nine of the flat datatypes the RDT operation is literally the sequential operation; the state carries invisible bookkeeping (tags, per-replica slots) and the spec is the evident program on the user-visible view.

- **Counter, increment-only counter.** Spec state  $\mathbb{Z}$  (resp.  $\mathbb{N}$ ); increment is  $n \mapsto n + 1$ . The fold *is* the history length (`counter_seq_sound`, `ioc_seq_sound`).
- **PN-counter.** Spec: `#inc - #dec (pnSpec)`; `pn_seq_sound`.
- **Grow-only set and map; conditioned G-set.** Spec: accumulation;  $x$  is a member iff some operation added  $x$  (`goset_seq_sound`, `gomap_seq_sound`, `gsetcond_seq_sound`).
- **OR-set, plain and efficient.** Spec state: a plain set; `add e` inserts, `rem e` deletes (`orSpecStep`). The tag machinery (fresh  $(t, e)$  stakes; remove filters every tag of  $e$ ) is invisible to one replica: the element view of the fold is the naive program (`orset_seq_sound`, `orsete_seq_sound`), a fold homomorphism, no invariant needed.
- **Enable-wins flag.** Spec state: one boolean; `enable`  $\mapsto$  true, `disable`  $\mapsto$  false (`ewSpecStep`); `ewflag_seq_sound`.
- **Add-wins priority queue.** A two-part spec pins both state components: membership behaves as the plain add/remove set with `inc` inert (`awpqSpecStep`, `awpq_mem_seq_sound`), and the increment component is a grow-only log of exactly the issued increments (`awpq_inc_log_sound`).

### 25.2 Gap: arbitration by stamp instead of program order

**LWW and FWW registers.** The sequential register is `write v` with *program order* deciding who wins; the spec programs are “the last write’s value” (`lwwSpecFold`) and “the first write’s value” (`fwwSpecFold`). The RDT operation carries its Lamport stamp *as the arbitration key in the payload* (a max- resp. min-semilattice on stamped writes), because concurrent replicas have no shared program order to appeal to. The gap is a transfer of ordering authority from control flow to data, and the honesty condition is precisely its repair: **stampsMono** (stamps strictly increase along the history, the sequential Lamport condition), under which the arbitration key

agrees with program order and `lww_seq_sound` / `fww_seq_sound` recover last- and first-write semantics. Without `stampsMono` the statements are false, and should be: a replica that back-dates a write is asking LWW to disagree with program order.

### 25.3 Gap: overwrite by snapshot instead of “everything”

**Multi-valued register.** The sequential write  $v$  implicitly overwrites everything before it. The RDT operation is `write`  $(v, O)$  where  $O$  is a **prepare-time snapshot**: the write-stamps the issuer currently sees, named explicitly so that a *concurrent* write (absent from  $O$ ) survives, which is the point of an MVR. The sequential spec is still the plain last-write register (`mvrSpecFold`). Honesty (`mvrOK`): the stamp is fresh and  $O$  lists *exactly* the issuer’s visible stamps, the formal form of “I overwrite precisely what I can see.” Under it, `mvr_seq_sound`: the visible-value view of the fold is the last write. The proof needs the campaign’s first genuinely discovered invariant, `mvr_over_tags`: every overwritten stamp was staked, which is what makes a fresh stamp provably not-yet-overwritten.

### 25.4 Gap: dequeue() becomes delete-what-I-saw

The **mergeable queue**, the cleanest specimen (Figure 12). The sequential queue is

$$\text{enq } v : S \mapsto S \mathbin{++} [v] \qquad \text{deq}() : S \mapsto \text{tail}(S).$$

The RDT operation is `deq`  $t$ : remove *by tag*,  $t$  being the tag of the element the issuing replica read as the head of its own queue; the implementation filters  $t$  out of the whole list, wherever it sits. The spec keeps the sequential shape, `tail`, blind to  $t$  (`qSpecStep`). Honesty (`qOK`) is witness faithfulness: the fold at the issue point is literally  $(t, v) :: \text{rest}$ . The invariant `q_tags_nodup` (fresh stakes stay duplicate-free) makes filter-by-tag remove exactly the head, and `queue_seq_sound` closes the loop: delete-what-I-saw implements pop-the-head. Its axiom audit is the leanest in the campaign:  $\{\text{propext}, \text{Quot.sound}\}$ , no choice anywhere. The same witness discipline is what the convergence side already consumed (`qHonest_of_applicable`); nothing was invented for the occasion.

### 25.5 Gap: insert-at-index becomes insert-after-who-I-saw

**The embedded-chain RGA.** The sequential text buffer is `insert`  $(i, e)$  and `delete`  $(i)$ , index-addressed. The RDT operation is `ins`  $(e, \pi, a)$ :  $a$  is the *id* of the element the issuer saw at position  $i - 1$  (its anchor; the sentinel 0 for the head), and  $\pi$  is the anchor’s stored coordinate as the issuer read it (the proof-carrying prefix); `del`  $(x)$  names the id seen at position  $i$ . The spec program is the buffer in id-clothing (`eSpecStep`): front-cons for  $a = 0$ , otherwise splice immediately after  $a$ ’s entry (`eInsAfter`); delete filters by id. On a sequential history the anchor id determines the index uniquely, so this *is* the positional program with positions named by their occupants. Honesty (`eSeqOK`): stamps exceed all prior insert stamps, and each operation is applicable at the current fold, i.e. the anchor is live and carries exactly  $\pi$ . The theorem (`embed_seq_sound`, §26) is the strongest form in the campaign: the canonical sorted state equals the spec buffer *record for record*. The same spec, verbatim, is what the rehoming RGA is refuted against (§27); its gap is not the witness (the rehoming op carries one too, plus a path) but the implementation moving *other* elements’ referents on delete.

## 25.6 Gap: one mark becomes a boundary pair

**Fused Peritext.** The sequential marked-text editor has an *atomic*  $\text{mark}(i, j, m)$ . The fused RDT has no such operation: a mark is **two** ordinary inserts of boundary elements  $\langle m \rangle$  and  $\langle /m \rangle$  sharing a fresh mark id, anchored at the range ends; mark removal is two deletes. An atomic positional operation is traded for a *pair* of witnessed inserts, which is the atomicity horn of the mark-positioning trilemma, accepted knowingly. The spec therefore lives at the token level: the editor is the §25.5 buffer over  $\text{char} \oplus$  boundary entries plus one left-to-right walk maintaining the open-mark set (a start boundary opens, the matching close removes, characters display with what is open). `peritextEmbed_seq_sound` says the datatype’s render is that editor’s screen, id for id in the `_ids` variant; the range-level reading is recovered by the positional intent theorems (`render_id_active_iff_between`: a character carries mark  $m$  iff it sits, in reading order, strictly between  $m$ ’s boundaries). The pair encoding is also why the character/boundary distinction is load-bearing in the delete theorem: deleting a *character* re-formats nothing (`renderIds_del`), while deleting a *boundary* is mark removal and must re-format its span (machine-checked necessary: `boundary_delete_breaks_filter_eq`).

## 26 The proofs: four tiers, three kinds of invariant

The campaign ran easiest to hardest, and the difficulty gradient turned out to be a gradient of invariant *kinds*.

**Tier 1 (eleven flat datatypes), `SeqSpec_Flat.lean`.** Mostly view homomorphisms, no invariant at all. The two exceptions need *identity hygiene*: facts about tag bookkeeping that hold on every reachable single-replica state and are exactly what the induction consumes. For the MVR, `mvr_over_tags` (overwritten stamps are staked); for the AWPQ, duplicate-freedom of stakes.

**Tier 2 (the mergeable queue), `MergeableQueue_SeqSpec.lean`.** One identity-hygiene invariant, `q_tags_nodup`, as described in §25.4.

**Tier 3 (the embedded-chain RGA), `EmbedRGA_SeqSpec.lean`.** The invariant becomes *geometric*. The spec splices an insert immediately after its anchor; the datatype sorts records by birth-chain coordinates; the bridge is the **adjacency lemma** (`chainBefore_snoc_iff`, Figure 13): a fresh insert’s chain  $c_a ++ [\delta]$  sits directly after its anchor’s chain  $c_a$  in display order, because the fresh timestamp gap  $\delta$  exceeds every delta any existing chain hangs off  $c_a$ , a fact that costs only sum-telescoping (a chain’s deltas sum to its id, and every existing id is below the fresh stamp). The sorted canonical state then *is* the spec buffer, record for record (`embed_seq_sound`). Two transports come for free. The published tombstoned RGA is read-equivalent to the embed on honest event sets (the compaction theorem, §21), so it inherits the buffer spec: `rga_seq_read_eq_buffer` and element-level `rga_seq_read_element` (`EmbedRGA_SeqSpec_RGA.lean`, a standalone build target because the two RGA models share top-level names). And the sided generalization simulates the one-sided instance on all-R histories unconditionally (`sFold_liftOp`), so the buffer spec transports to the sided datatype’s all-R fragment at zero proof cost; the full two-sided theorem has since landed as `sided_seq_sound`.

(`SidedRGA_SeqSpec.lean`): a sentinel-rooted buffer where an R-insert splices immediately after its anchor and an L-insert immediately before it, via the two sided adjacency lemmas (`schainBefore_snocR_iff`, `snocL_iff`, axioms `{propext, Quot.sound}`).

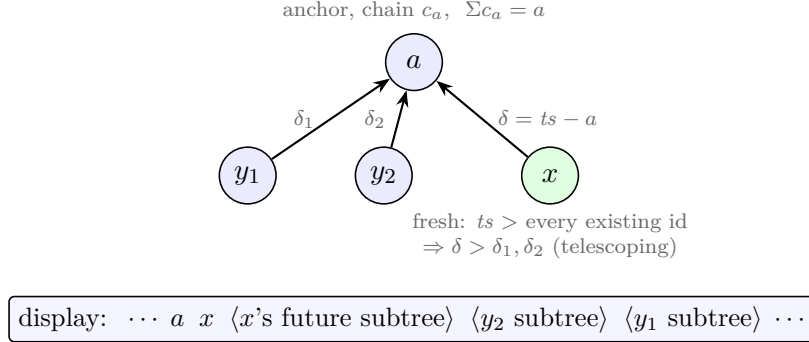


Figure 13: The adjacency lemma, the geometric heart of tier 3. Siblings display newest-first; a fresh insert’s delta beats every existing child’s because deltas telescope to ids and every existing id is below the fresh stamp. Hence the newcomer lands *immediately* after its anchor: exactly what the sequential buffer’s splice does.

**Tier 4 (fused Peritext), `PeritextEmbed_SeqSpec.lean`.** The invariant kind is *composition*, and that is the finding: tier 3’s buffer soundness, made payload-generic, already says the fold’s (id, element) sequence is the naive buffer, and the datatype’s render and the editor’s screen are literally the same pure fold over that sequence (`peritextEmbed_seq_sound`). No new invariant was needed. This is also why the tier had to wait for the re-based Peritext: against the rehoming kernel the same statement is false (§27).

## 27 The refutations

The statement shape is falsifiable per datatype, and three refutations landed, all machine-checked, each with its honesty guards discharged on the witness so the failure is never an artifact of a malformed trace.

**The rehoming RGA fails at do (`rehoming_seq_refuted`).** Figure 14. Four operations on one replica, honest in the datatype’s own terms (every op `accurate` and `fresh_ts` at its state): build  $\langle b, a, c \rangle$  with  $c$  under  $a$ , then delete  $a$ . The naive buffer keeps survivor order,  $\langle b, c \rangle$ ; the rehoming `do` re-homes  $c$  to the root, where the newest-first sibling tiebreak leapfrogs it over  $b$ : the read is  $\langle c, b \rangle$ . The should-pass half (`seq_agrees_before_delete`) shows both sides agree on the delete-free prefix, isolating the delete as the exact point of departure. The design’s convergence capstone is untouched: it converges, to its own reordering fold. This refutation is what demoted the design from the repository’s canonical seat.

**Fused Peritext on the rehoming kernel fails at the render (`fused_delete_reformats_survivor`).** The same reorder lifted to  $\text{char} \oplus$  boundary: deleting a plain character re-homes a survivor past a mark boundary, and an untouched character changes formatting, on one replica. This upgraded the re-base of Peritext onto the embed

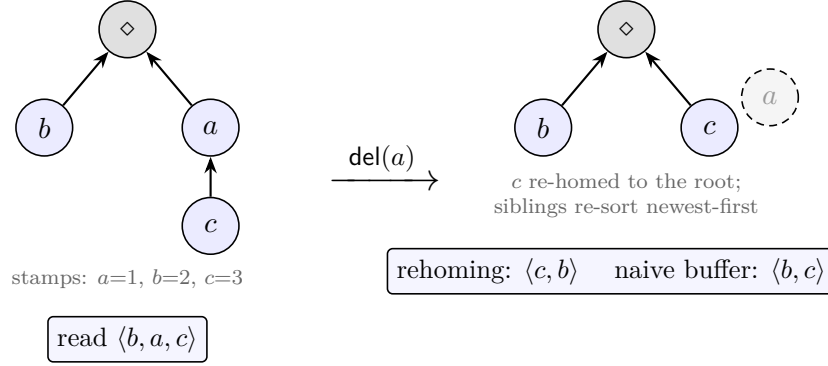


Figure 14: The four-op refutation. One replica, no merge. Deleting  $a$  swaps the surviving  $b$  and  $c$  in the rehoming RGA’s read; the naive buffer keeps their order. The embedded-chain RGA reads  $\langle b, c \rangle$  on the same trace:  $c$ ’s immutable coordinate still carries  $a$ ’s chain as pure geometry, so nothing moves.

kernel from housekeeping to a correctness fix. On the embed kernel the corresponding *general* theorem holds: `renderIds_del` (axioms  $\{\text{propext}, \text{Quot.sound}\}$ ): deleting a character leaves every other character’s rendered formatting bitwise untouched, because character entries are open-set-neutral and delete is a filter on an order-canonical state.

**Shesha passes sequentially and fails at the merge.** The mirror image. Shesha’s single-replica reads are the naive linked list (`sequential_soundness`, kernel-clean), and it uniquely preserves survivor order under delete among the tombstone-free designs of its generation (`delete_preserves_survivor_order`). Its *merge* is machine-checked non-RA-linearizable (`Shesha_Rows_Refuted`): at an honest reachable configuration the three-way merge splits a deleted node’s children around a concurrent sibling, producing a read that is the fold of no delivery-respecting linearization.

## 28 The two-axis separation

Together the refutations witness that do-faithfulness and merge-faithfulness are *independent* axes (Figure 15): the rehoming family occupies merge-yes/do-no, Shesha do-yes/merge-no, and the embedded-chain family shows the top corner is achievable, by one design idea in both sequence datatypes: immutable birth coordinates, so that deletion moves nothing and the merge decides nothing.

## 29 What composition does not give

It is tempting to compose the guarantees: RA-linearizability says every version is  $\text{fold}(\rho)$  for some delivery-respecting  $\rho$  over the event union; the campaign says folds of sequentially honest histories equal the spec program. May one conclude that every merged document is the naive program’s output on *some* ordering of everyone’s operations? **No**, and the failure is precise.

The campaign’s theorems quantify over *sequentially honest* histories; an RA-linearizability witness is delivery-respecting but need not be sequentially honest on either count. Stamp



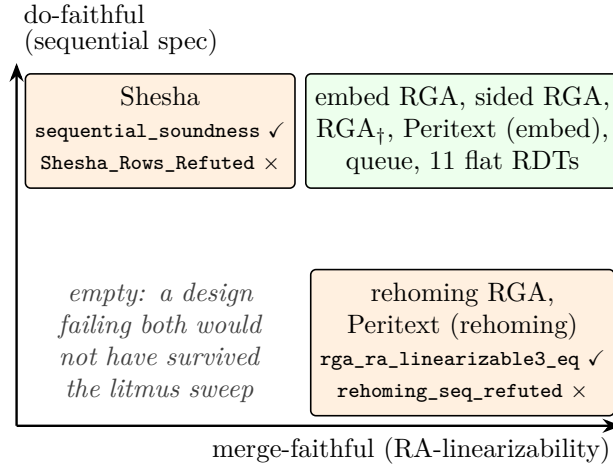


Figure 15: Every placement is a named kernel theorem. The two failure modes are orthogonal: converging to a reordering fold (bottom right) and folding correctly into a non-linearizable merge (top left).

monotonicity fails harmlessly: when the state is a function of the event set (fold-canonicity, `e_fold_canon`), any enumeration can be re-sorted. Applicability is the real corner (Figure 16): in the union of two branches, an insert’s anchor may be deleted by a *concurrent* operation carrying a smaller Lamport stamp, so in stamp-sorted order the delete precedes the insert, the anchor is dead at the insert’s prefix, and the naive buffer no-ops the splice, while the datatype (whose coordinates travel in the op) places the element correctly. Some interleavings of the union match the spec program and some do not.

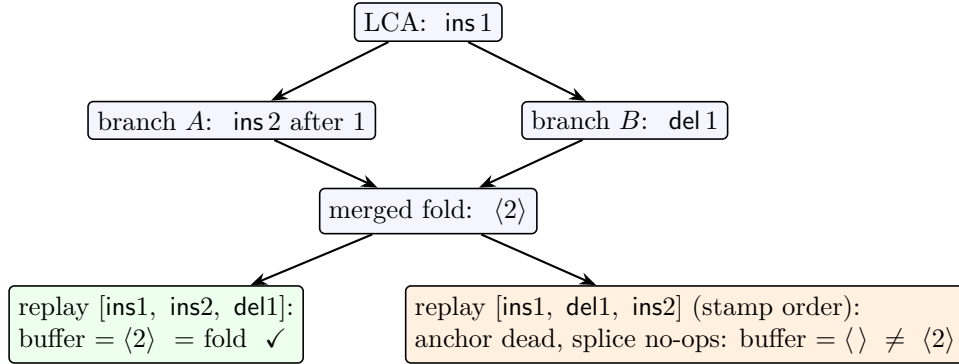


Figure 16: The dead-anchor corner. The merged fold keeps 2 (OR-set survival on immutable coordinates); whether a naive-buffer replay of the union agrees depends on which delivery-respecting order is chosen, and the stamp-sorted one can be wrong. This is the same corner the sequence-CRDT trilemma names for insert-after-already-deleted-anchor.

Whether a matching order always exists is genuinely open; it is recorded as Open Question 31.9 in the consolidated list.

Independently of the question, the composition that *is* sound and kernel-checked today: every version is the datatype’s fold of a delivery-respecting enumeration (RA-linearizability); the fold

is canonical per event set (`e_fold_canon`); each replica’s own contribution obeyed the sequential spec while it was being made (this campaign); and merge-level intent is delivered by separate, named theorems where it holds (delete-order preservation as sublist stability, subtree convexity as non-interleaving, read-equivalence to the published RGA as the compaction theorem). The campaign does not manufacture a merge-level user story; it pins the single-replica story so that merge-level claims have something sound to stand on.

## 30 The verified matrix

The anomaly-comparison matrix (task #64, `whiteboard/anomaly-matrix/`) ran six implementations and four production libraries through 25,917 merges per row, every cell backed by executed code. Its in-repo rows have since been upgraded from machine-run to kernel-checked; the table below is the matrix’s load-bearing core with each cell a Lean identifier. External libraries (Yjs, Automerge, Loro, list-positions) remain empirical by nature; their rows live in the matrix report unchanged.

| design                     | sequential spec (do)                                          | RA-lin (merge)                              |
|----------------------------|---------------------------------------------------------------|---------------------------------------------|
| embedded-chain RGA         | <code>embed_seq_sound</code>                                  | <code>embed_ra_linearizable3</code>         |
| sided RGA (two-sided)      | <code>sided_seq_sound</code>                                  | <code>sided_embed_ra_linearizable3</code>   |
| published RGA <sub>†</sub> | <code>rga_seq_read_eq_buffer</code>                           | <code>rgaWithTombstones_ra_lin...eq</code>  |
| Peritext (embed)           | <code>peritextEmbed_seq_sound</code>                          | <code>peritextEmbed_ra_linearizable3</code> |
| mergeable queue            | <code>queue_seq_sound</code>                                  | <code>queue_ra_linearizable3</code>         |
| eleven flat RDTs           | <code>*_seq_sound</code>                                      | <code>*_ra_linearizable3_eq</code>          |
| rehoming RGA               | <code>refuted: rehomining_seq_refuted</code>                  | <code>rga_ra_linearizable3_eq</code>        |
| Peritext (rehoming)        | <code>refuted (render): fused_delete_reformats_surv...</code> | <code>peritext_ra_lin...eq</code>           |
| Shesha                     | <code>sequential_soundness</code>                             | <code>refuted: Shesha_Rows_Refuted</code>   |

Every positive theorem above is kernel-clean (axioms  $\subseteq \{\text{propext}, \text{Classical.choice}, \text{Quot.sound}\}$ ; the queue’s and `renderIds_del`’s audits omit choice). Refutations by concrete witness follow the SPOT convention (`native_decide`, adding the compiler axioms); the refuted *statements* are quantified properties whose honesty guards are discharged on the witness.

## Part IV

# Open questions and the mechanization map

## 31 Open questions

The consolidated list: the framework questions of the metatheory (Part I), the campaign’s successor question (Part III), and two questions raised by the FugueMax arc (Part II). One resolved question is kept at the end in compressed form, because the later parts depend on its correction.

**Open Question 31.1** (Is there a rely-guarantee program logic for MRDTs?). *The framework already carries the skeleton of a verification-condition logic: the signature-plus-conditioning is the “program”, RA-linearizability up to  $\approx$  is the judgment, the eight VCs are local proof obligations, the metatheorem is their soundness theorem, and the product  $\otimes$  with `JoinLemma3At` is a frame/composition rule (the coupling hierarchy L0–L3 stratifying it by interaction strength,*

as disjoint-vs-shared reasoning does in separation logic). The conditioning layer is specifically *rely-guarantee*: honest delivery is the *rely* (an assumption on other replicas’ and clients’ steps), convergence/safety up to  $\approx$  is the *guarantee*, merges are environment steps, and the “stable under concurrent honest extension” condition the safety metatheorem turns on (§13: the bounded counter succeeds and BudgetCart/FWW fail on exactly it) is the *rely-guarantee stability side-condition*, arrived at independently. What is missing to make it a program logic proper is a syntax: a language building *do/merge* from primitives (a per-replica counter cell, OR-set membership, an LWW/FWW max/min cell, a sequence node) and combinators (the product  $\otimes$ , the indexed map, the payload-parametric wrap of *ORSetCore*), each carrying its local VC contribution and its join-species, such that well-formedness entails RA-linearizability by construction — so that writing an MRDT and proving it converge become one act, as separation logic made heap-program verification syntax-directed. The semantic combinators already exist and are each separately proven sound; what is open is the grammar together with a single soundness theorem for the grammar (not one per combinator). This question subsumes the two framework-theory questions below: CDVC3-derivability (31.2) asks whether the rule set is minimal, and the structure theorem asks to classify the primitives.

**Open Question 31.2** (CDVC3-derivability). *Is CDVC3 derivable from VCs 1–7 (plus the unconditional delta laws where they hold)? Specializing the merge to an LCA-blind lattice join — the CRDT case — the question closes exactly: there, the corresponding causal-delta bound is equivalent to the Join Lemma and no strictly weaker bridge VC exists, with both attack routes characterized — the positive induction is circular at two identified case shapes, and the countermodel space is narrowed to non-atomic lattice states (all of this mechanized, in *Sal/CRDTs/Metattheory/*). The ternary question, by contrast, is now **closed**: CDVC3 is an independent rule of the ternary bundle (`cdvc3_not_derivable_from_core_delta`, *Refutations/CD\_Not\_Derivable\_Ternary.lean*, kernel-clean) — there is a *ConditionedMRDTSig* (*AWSetF3*, the LCA-blind lift of the binary inflation-separator, with a deflationary *rem*) satisfying every *CoreVCs3* and *DeltaVCs3* clause yet falsifying CDVC3, so  $\text{CoreVCs3} + \text{DeltaVCs3} \not\models \text{CDVC3}$  (and, via *joinLemma3\_iff\_cdvc3*,  $\not\models$  the Join Lemma). The reason ternary closes where binary ( $b''$ ) stays open is instructive: CDVC3’s  $\sqsubseteq$ -half demands update inflation ( $A \sqsubseteq \text{update } A e$ ), which the binary *LatticeVCsPlus* ships as a separate axiom but the ternary *DeltaVCs3* honestly omits — so the inflation-free delta laws cannot back CDVC3, and a deflationary update refutes it. The eight-VC bundle is therefore **minimal at CDVC3**: the causal-delta rule is a genuine independent axiom, not a derived lemma (a load-bearing point for the rule-basis of Open Question 31.1)*

**Open Question 31.3** (structure theorem). *Empirically, the unconditional delta contract holds exactly on the group and lattice classes. Is every unconditional-DeltaVCs3 merge a torsor-like group  $\otimes$  lattice combination? (The naïve two-sorted umbrella  $\text{mergeL}(l, a, b) = a \boxplus (b \boxplus l)$  does not derive the contract uniformly, so the question is genuinely open.)*

**Open Question 31.4** (the automation layer). *The published catalogue’s seventeen induction-scheme VCs were an SMT-facing Skolemization of feasibility, aimed at the wrong target. Design the analogous per-data-type induction scheme whose induction implies CDVC3 and the feasible delta laws — restoring push-button discharge, now of VCs that actually imply RA-linearizability. The observed uniformity of the discharges ( $\sigma$ -characterization by sandwich invariant + absorber trichotomy) suggests the scheme exists.*

**Open Question 31.5** (criss-cross merges and virtual LCAs). *The transition system gates Merge on the existence of an LCA (§4.1); criss-cross configurations are reachable, and every version in them linearizes, but their heads cannot merge — the model is silent exactly where git reaches for its recursive strategy (merge the maximal common ancestors into a virtual base, then merge with that). The metatheory looks ready for the extension: the ternary Join Lemma is stated over backward-closed event sets, and  $E_1 \cap E_2$  needs no witnessing store version. What remains is a virtual-LCA Merge rule plus one per-data-type question: do the eight VCs imply that the recursively computed base equals  $\sigma(E_1 \cap E_2)$ ? For the counter this is inclusion–exclusion once more — exact iff the maximal common ancestors jointly cover the intersection, itself a candidate store invariant; for the OR-set, a pointwise Boolean check. A positive answer closes the one visible gap between the model and practice.*

**Open Question 31.6** (rc-expressiveness: chains, LWW, and merge-recoverability). *The catalogue’s rc relations are all star-shaped: the conflicting pairs are cross-class (rem/add; add/checkout), so every edge runs from a class that is never second to a class that is never first. This is forced: the update layer’s directionality law (`rc_non_comm_directional`) orients every non-commuting concurrent pair, while its companion `no_rc_chain` forbids length-2 paths, and any orientation of a self-conflicting class (concurrent writes; concurrent claims of one slot) contains one. So total-order policies — last-writer-wins first among them — are inexpressible in the rc mechanism as constrained by the VC set. Two questions, one of them already closed at its boundary. (i) Would a chain-tolerant generalization (the concurrent arm of lo as a transitive tiebreak) be sound? The semantics is coherent — with Lamport clocks the composite of vis and any timestamp- or priority-oriented tiebreak is acyclic, and canonical states remain well-defined — so the restriction appears to be an artifact of the swap-flavored proof machinery (`cond_comm_lift` cannot bubble past two consecutive rc edges), not of the theorem. The decisive experiment: reprove the swap VC and the causal-delta peel over a transitive tiebreak; we expect the swap route to break and the canonical-state route (which peels a global maximum, and a total order has one everywhere) to survive. (ii) What would it buy? Not a metadata-free LWW register: whatever the order arbitrates, the converged outcome must be produced by `mergeL(l,a,b)` — a function of three states — and for a plain-value LWW two executions present identical state triples with opposite timestamp orders, so no such function exists (`lww_merge_needs_timestamps`, *Refutations/LWW\_Merge\_Needs\_Timestamps.lean*, kernel-checked). The arbitration key is forced into the state by the merge; once there, writes commute and rc is moot — which is why every LWW implementation stores its timestamp. What a chain-tolerant arm could buy: metadata-free priority policies across three or more op classes (outcomes state-recoverable, orientations chained). The general principle the kill-test isolates: rc is the mechanism for conflicts whose outcome is recoverable from branch states; clique conflicts pay for their arbitration key in state, and chains would extend rc only within the recoverable class.*

**Open Question 31.7** (Causal canonicity under general rc). *The generic safety metatheorem consumes causal canonical witnesses — enumerations linearizing  $\xrightarrow{\text{vis}}$  and respecting  $\text{lo}^E$  jointly. With every concurrent pair unordered (Either), the two discharge species of §13 suffice. Under a general rc the existence of a joint linearization is open: a cycle would need at least two rc-edges separated by  $\xrightarrow{\text{vis}}$ -paths — a single rc-edge closed by a  $\xrightarrow{\text{vis}}$ -path contradicts concurrency via transitivity, and a non-commuting first  $\xrightarrow{\text{vis}}$ -step is an absorber killing the edge — and `no_rc_chain` forbids adjacent rc-edges but not  $\xrightarrow{\text{vis}}$ -separated ones. *CausalCanonical* is therefore a hypothesis with two discharge lemmas rather than a theorem. The budget cart (*BudgetCart*,*

*MRDT\_Instances/BudgetCart/*) has now forced the question — and widened it: its convergence discharges along the *OR*-set route, but the ungated safety obligation *SafetyStepOn* is false for it, by a two-event refutation — *CausalFold* pins only  $\xrightarrow{\text{vis}}$ -respect, and the fold of a causal past containing a concurrent same-item *add/rem* pair is enumeration-dependent (the orders disagree exactly where *add*-wins should arbitrate). So for *rc*-nontrivial datatypes the honesty fold needs *rc*-orientation too: the right notion is folds along jointly  $\xrightarrow{\text{vis}}$ - and  $\text{lo}^E$ -respecting enumerations, for both the version witnesses (*CausalCanonical*) and the causal-past folds (*CausalFold*). The cart’s safety theorem is delivered gated (*bcart\_version\_inv\_gated*, with the missing transfer named explicitly as *BCartSpendMono*); closing the gate is this question.

**Open Question 31.8** (Does conditioning compose? — convergence: yes, now a theorem). The catalogue is built one data type at a time; practice builds data types from data types — the canonical case is the key-value map whose values are themselves MRDTs (the certified-MRDT line’s own composite). The factored framework suggests composition should happen at the *JoinLemma3At* boundary — the map combinator would consume each value’s per-configuration join, whatever species produced it, so a map of queues composes as readily as a map of counters — with a small once-only kit: folds and configurations project per key, cross-key pairs commute so  $\text{lo}^E$  localizes, and per-key joins re-interleave. Conjecturally the coupling strength stratifies what survives composition: payload parametricity (free), the indexed product = the grow-only-keys map (a theorem to prove), read coupling — one layer’s *app* reads another’s state, as in a budgeted cart — (convergence composes; safety needs a monotone-transfer lemma), uniform coupling — key removal, reaching into the value layer the same way for every value type (one generic obligation: a reset element and a *remove || v-op* policy) — and ad-hoc effect coupling (rich-text marks anchored in a sequence), where nothing is expected to compose automatically. The convergence half is now mechanized (*Metatheory/Product.lean*): the binary heterogeneous product  $D_1 \otimes D_2$  composes at exactly the *JoinLemma3At* boundary — cross-component pairs commute definitionally, so  $\text{lo}^E$  has no mixed edges and the glued join witness is a plain concatenation  $\iota_1 \rho^1 ++ \iota_2 \rho^2$  (*joinLemma3At\_prod*; composite metatheorem *prod\_ra\_linearizable3\_of\_honest\_reach*, axioms *propext*, *Quot.sound*; consumability demo: a queue  $\otimes$  counter capstone with zero bespoke proof, *MRDT\_Instances/ProductDemo/*). Two boundary findings from the pen-and-paper pass (*Development/COMPOSITION\_PENPAPER.md*): reachability does not project (a second-component apply is a version-duplicating stutter for the first’s projection), so finished capstones never compose — only per-configuration certificates do, which retroactively forces the factored interface of §13; and two-sided causal-witness re-interleaving is refuted (a realizable four-event cross- $\xrightarrow{\text{vis}}$  cycle), with the sound repair — pin one side, re-sort the other — covering products with at most one *rc*-nontrivial component. The safety and  $\approx$  kits are also mechanized: honesty, *SafetyStep*, and one-sided causal witnesses compose (*Metatheory/Product\_Safety.lean*: the pinned-extension lemma, *safetyStepOn\_prod*, *causalCanonical\_prod\_of\_one\_sided*, composite *prod\_version\_inv\_on\_of\_one\_sided*), and the eq-quotient layer lifts at the pragmatic cut  $\approx_2 =$  (*Metatheory/ProductEq.lean*: every *GenericEqQuotient* obligation componentwise, *eqJoinLemma3C\_H\_prod* glued by concatenation with no congruence chasing, capstone *prod\_ra\_linearizable\_up\_to\_eq\_H* — one quotiented component, one flat, exactly the Peritext shape, with the quotiented side’s reachability supplies as premises per the memo’s cost-center analysis). Still open: the general  $\approx_1 \times \approx_2$  lift (no new ideas, double the plumbing — deferred until a second quotiented component exists), the indexed (map-of-MRDTs) generalization, and the L2.5/L3 coupling boundaries, where composition breaking is expected to be as informative as where it holds.

**Open Question 31.9** (the sequential witness at merges). *On every honestly reachable configuration of the embedded-chain RGA, does the event union admit a delivery-respecting enumeration that is also sequentially honest, so that the merged read is the naive buffer’s output on it? Caution from adjacent territory: for the rehoming design the analogous “order dependency edges first” discipline is refuted (the *DepComp* refutation, via rehoming non-transitivity). The embed’s immutable coordinates remove that refutation’s mechanism, but no proof exists in either direction.*

**Open Question 31.10** (the display-to-traversal theory, and the first mechanized maximal non-interleaving). *The FugueMax arc (§21) formalized the Weidner–Kleppmann maximal-non-interleaving Definition 4 over reachable configurations, refuted it for the Fugue-with-recency policy (the paper’s own Fugue-vs-FugueMax separation, machine-checked), and proved its condition (3) for the FugueMax kernel variant (`fuguemax_same_origin_low_first`). What remains of the full Theorem 9 is the display-to-traversal theory (task #88): relating the kernel’s displayed order to the tree traversal the definition quantifies over. Completing it would yield the first fully mechanized maximal non-interleaving theorem for a sequence datatype.*

**Open Question 31.11** (the retention lower bound for tombstone-free maximal non-interleaving). *The FugueMax kernel variant realizes its sibling order by embedding the mint-time successor’s key into contested *R*-entries, at  $\Theta(|\text{origin key}|)$  coordinate cost where plain Fugue pays zero (§21). Is that retention a lower bound: must any tombstone-free design achieving maximal non-interleaving carry the right origin’s ordering information in its keys? The question has a Myhill–Nerode shape (task #89): distinguish states that demand different verdicts, and count what any key scheme must remember. It is the entropy law of §15 meeting the ordering intent.*

**Resolved: the self-referential-spec limit.** RA-linearizability certifies that a version’s state is the fold of some delivery-respecting linearization of its events: convergence to the datatype’s *own* do-fold, with that fold as the reference. It says nothing about whether the fold’s single-replica semantics is the intended one; a defect in *do* appears identically in the concurrent state and its fold, and is certified as agreement. Independent *intent* specifications are therefore a separate obligation, not a corollary of convergence. The question was resolved systematically by the sequential-specification campaign (Part III), which supplies the independent specification for every production RDT; its refutations also correct the earlier reading of the sharpest instance. The survivor reorder exhibited by the campaign’s four-op trace (§27) is not intrinsic to tombstone-freedom, it is intrinsic to rehoming: the embedded-chain RGA is tombstone-free, satisfies the sequential list specification in the strongest form (`embed_seq_sound`: the canonical state is the naive buffer, record for record), and preserves delete order (`eUpdate_del_sublist`). The successor question is Open Question 31.9.

## 32 Mechanization map

Everything above is mechanized in Lean 4 (Mathlib; axioms: `propext`, `Classical.choice`, `Quot.sound`; no `sorryAx` anywhere in the cited chains; refutations by concrete witness follow the SPOT convention, `native_decide` adding the compiler axioms) in the Sal repository. Files are relative to `Sal/ConditionedMRDTs/` unless they carry an explicit `Sal/` prefix.



| Artifact                                               | Lean                                                                | File                                                                 |
|--------------------------------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------------|
| <i>Part I: framework and metatheorems</i>              |                                                                     |                                                                      |
| signature, ternary $\text{lo}$                         | MRDTSig, ConditionedMRDTSig                                         | Framework/MRDTSig.lean                                               |
| store, DAG, transitions                                | Configuration, Step3                                                | Framework/ExecutionModel.lean, Metatheory/LCA_Lemma.lean             |
| Lemma 4.1                                              | lca_events_of_storeInv                                              | Metatheory/LCA_Lemma.lean                                            |
| $\sigma/\text{lo}^\sigma$ layer (merge-independent)    | UpdateVCs, IsCanonicalState                                         | Framework/Sigma_LoOn3.lean                                           |
| the eight VCs                                          | CoreVCs3CD, FeasibleDeltaVCs3, CDVC3                                | Framework/VC_Set.lean                                                |
| Theorem 8.1 + bridges                                  | ra_linearizable_of_core_feasible_cd3                                | Metatheory/Adequacy.lean                                             |
| defeater vs. 2-OP (§4.2)                               | no_inter_lca_2op_rm_peel_of_defeater                                | Refutations/InterLca2op_Defeater_Arbiter.lean                        |
| closure-indexed contract (§8)                          | JoinLemma3C, no_proper_block                                        | Metatheory/JoinLemma3C.lean                                          |
| closure-indexed adequacy + instances                   | ra_linearizable3_of_joinC, ConditionedContract                      | Metatheory/ConditionedContract.lean                                  |
| all 9 flat discharges via the contract                 | *_adequate_viaContract (9)                                          | Development/AllMRDTS_viaContract.lean                                |
| per-configuration join hook                            | JoinLemma3At, goodConfig3_merge_at                                  | Framework/VC_Set.lean, Metatheory/Adequacy.lean                      |
| honest-reachability metatheorem                        | HonestReach, ra_linearizable3_of_honest_reach                       | Metatheory/HonestReach.lean                                          |
| generic honesty shape                                  | GenHonest, AppHonest                                                | Metatheory/GenHonest.lean                                            |
| witness-class join lemmas                              | JoinLemma3AtW                                                       | Metatheory/WitnessClass.lean                                         |
| generic safety (causal witness)                        | SafetyStep, version_inv_of_causal_canonical                         | Metatheory/GenericSafety.lean                                        |
| escrow metatheorem                                     | Measured, escrow_version_inv                                        | Metatheory/EscrowSafety.lean                                         |
| product composition (raw kit)                          | JoinLemma3At_prod, prod_ra_linearizable3_of_honest_reach            | Metatheory/Product.lean                                              |
| product safety kit                                     | safetyStepOn_prod, prod_version_inv_on_of_one_sided                 | Metatheory/Product_Safety.lean                                       |
| product $\approx$ -lift                                | eqJoinLemma3C_H_prod, prod_ra_linearizable_up_to_eq_H               | Metatheory/ProductEq.lean                                            |
| observational quotient $D \mapsto D_\approx$           | QSig, qapplicable                                                   | Metatheory/GenericEqQuotient*.lean                                   |
| canonical-witness layer, raw- $\approx$ target         | IsRALLinearizable3Eq, RA_linearizable_up_to_eq_H                    | Metatheory/GoodConfig3H.lean                                         |
| flat collapse of the framework                         | eqJoinH_of_joinC, flat_ra_linearizable3_eq                          | Metatheory/FlatGeneric_Bridge.lean                                   |
| LWW merge needs timestamps (OQ 31.6)                   | lww_merge_needs_timestamps                                          | Refutations/LWW_Merge_Needs_Timestamps.lean                          |
| CDVC3 independent of core+delta (OQ 31.2)              | cdvc3_not_derivable_from_core_delta                                 | Refutations/CD_Not_Derivable_Ternary.lean                            |
| impossibility kill-tests                               | —                                                                   | Refutations/Impossibility.lean                                       |
| findings journal                                       | —                                                                   | Development/MRDT_METATHEORY_DRAFT.md                                 |
| <i>Part I: instances (Table 1)</i>                     |                                                                     |                                                                      |
| catalogue instance theorems                            | all instance theorems                                               | MRDT_Instances/ (one directory per RDT)                              |
| the flat instances, generically                        | *_ra_linearizable3_eq (11)                                          | MRDT_Instances/ (per-RDT)                                            |
| bounded-counter safety                                 | bc_version_inv, bc_value_nomneg                                     | MRDT_Instances/BoundedCounter/                                       |
| mergeable queue (direct join)                          | q_join_at, queue_ra_linearizable3                                   | MRDT_Instances/MergeableQueue/                                       |
| FWW register (payload arbitration)                     | fwv_version_min                                                     | MRDT_Instances/FWWRegister/                                          |
| BudgetCart (gated safety)                              | BCart_ra_linearizable3_eq, bcart_version_inv_gated                  | MRDT_Instances/BudgetCart/                                           |
| <i>Part II: the embedded-chain family</i>              |                                                                     |                                                                      |
| model-layer kernel                                     | rc_non_comm', display_iff_chainBefore, subtree_convex               | Sal/MRDTS/RGA_Embed/                                                 |
| embedded-chain RGA (queue route)                       | embed_ra_linearizable3, e_fold_canon                                | MRDT_Instances/EmbedRGA/                                             |
| Elias- $\delta$ inheritance corollary                  | (capstone at the $\delta$ -flip)                                    | MRDT_Instances/EmbedRGA/EmbedRGA_EliasDelta.lean                     |
| compaction theorem                                     | rga_read_eq_embed_read                                              | MRDT_Instances/EmbedRGA/EmbedRGA_ReadEquiv.lean                      |
| compaction order core                                  | visible_lt = chain-lex                                              | Sal/MRDTS/RGA_Embed/RGA_Embed_ReadEquiv.lean                         |
| sided chain-lex kernel                                 | sdisplay_iff_schainBefore, schainBefore_liftR                       | Sal/MRDTS/RGA_Embed/Sided_ChainLex.lean                              |
| sided RGA + per-policy intent                          | sided_embed_ra_linearizable3, sFold_liftOp                          | MRDT_Instances/SidedRGA/                                             |
| Fugue/FugueMax vs. W-K Def. 4                          | fugue_not_maximally_noninterleaving, fuguemax_same_origin_low_first | MRDT_Instances/SidedRGA/, Sal/MRDTS/RGA_Embed/SidedMax_ChainLex.lean |
| Peritext on the embed kernel                           | peritextEmbed_ra_linearizable3, renderIds_del                       | MRDT_Instances/Peritext_Embed/                                       |
| <i>Part III: the sequential-specification campaign</i> |                                                                     |                                                                      |
| tier 1: eleven flat RDTs                               | *_seq_sound (per RDT)                                               | MRDT_Instances/SeqSpec_Flat.lean                                     |
| tier 2: mergeable queue                                | queue_seq_sound                                                     | MRDT_Instances/MergeableQueue/MergeableQueue_SeqSpec.lean            |
| tier 3: embedded-chain RGA                             | embed_seq_sound                                                     | MRDT_Instances/EmbedRGA/EmbedRGA_SeqSpec.lean                        |
| tier 3, two-sided: sided RGA                           | sided_seq_sound                                                     | MRDT_Instances/SidedRGA/SidedRGA_SeqSpec.lean                        |
| tier 3 corollary: RGA <sub>1</sub>                     | rga_seq_read_eq_buffer                                              | MRDT_Instances/EmbedRGA/EmbedRGA_SeqSpec_RGA.lean (standalone)       |
| tier 4: fused Peritext                                 | peritextEmbed_seq_sound                                             | MRDT_Instances/Peritext_Embed/PeritextEmbed_SeqSpec.lean             |
| render fix theorem                                     | renderIds_del                                                       | MRDT_Instances/Peritext_Embed/PeritextEmbed.lean                     |
| negative: rehoming RGA                                 | rehoming_seq_refuted                                                | MRDT_Instances/RGA_Rehoming/RGA_SeqSpec_Refuted.lean                 |
| negative: rehoming Peritext                            | fused_delete_reformats_survivor                                     | MRDT_Instances/Peritext_Rehoming/Peritext_Read.lean §10              |
| negative: Shesha merge                                 | Shesha_Rows_Refuted                                                 | MRDT_Instances/Shesha/Shesha_Rows_Refuted.lean                       |

The update-side layer ( $\text{lo}^E$ , convergence,  $\sigma$ ) is merge-independent — it is stated over the update signature  $\langle \Sigma, \sigma_0, \text{do}, \text{rc} \rangle$  alone — and is shared with the binary (CRDT) specialization of the theory; in the repository it lives in `Sal/CRDTS/Metatheory/`, the directory that also holds the binary exactness results quoted in Open Question 31.2. The production data types of Table 1 are mirrored faithfully from `Sal/MRDTS/*` (encoding deviations documented per instance); their original 24-VC discharges are untouched.

The foundations live under `Framework/`, the metatheorems under `Metatheory/`, the machine-checked negative results under `Refutations/`, and the per-RDT conditioned instances under `MRDT_Instances/`; findings journals and superseded routes remain under `Development/`, which nothing imports. Each cited theorem is kernel-checked with axioms in  $\{\text{propext}, \text{Classical.choice}, \text{Quot.sound}\}$  (no `sorryAx`). They import one linearization file carrying two unrelated `sorrys` that the cited theorems do not transitively depend on; `Development/CONDITIONED_METATHEORY_PLAN.md` is their roadmap.